UNIVERSIDAD MAYOR

ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN E INFORMÁTICA

EVALUACIÓN DE LA EFICIENCIA DE YOLOV8 EN GPU, CPU Y RASPBERRY PI: CONVOLUCIONES ESTÁNDAR VS SEPARABLES

PROYECTO DE TESIS

Por

PABLO VICENTE REYES PINO

Tutor Dr. Anthony D. Cho

Santiago, Chile.
Abril 2026

Tabla de Contenidos

Índice de Figuras 3
Índice de Tablas 4

1. Introducción **7**
1.1. Descripción del Problema 7
1.2. Justificación 8
1.3. Objetivos 9
1.3.1. Objetivo General 9
1.3.2. Objetivos Específicos 9
1.4. Alcance del Proyecto 9
1.5. Hipótesis de Investigación 9

2. Marco teórico **10**
2.1. Historia y Evolución de la Detección de Objetos 10
2.2. Incremento en la Capacidad de Cómputo 11
2.3. Variantes de los Modelos de YOLO 14
2.4. YOLOv1 a YOLOv3 14
2.5. YOLOv4 a YOLOv5 14
2.6. YOLOv6 y YOLOv7 15
2.7. Aplicaciones de Modelos de Detección de Objetos 15
2.7.1. Vehículos Autónomos 15
2.7.2. Seguridad y Vigilancia 16
2.7.3. Industria y Manufactura 18
2.8. Introducción al Modelo YOLOv8 20
2.8.1. Impacto y Aplicaciones de YOLOv8 20
2.8.2. Backbone 21
2.8.3. Funciones de Activación en YOLOv8 22
2.8.4. Neck 22
2.8.5. Head 24
2.8.6. Componentes Avanzados 25
2.9. Convolución Separable 27

2.9.1. Convolución Depthwise . . . 27
2.9.2. Convolución Pointwise . . . 28
2.9.3. Ventajas de las Convoluciones Separables . . . 29
2.10. Comparativa entre Raspberry Pi y ESP32-CAM . . . 29
2.11. Razones para la Selección de la Raspberry Pi . . . 30

3. Metodología **31**
3.1. Diseño de Investigación . . . 32
3.2. Recolección y Preparación de los Datos . . . 32
3.3. Implementación del Modelo . . . 33
3.4. Adaptación propuesta para modelo YOLO v8 con capas convolucionales separables . 34
3.5. Entrenamiento y Evaluación . . . 34
3.5.1. Criterio de selección del número de épocas . . . 35
3.6. Métricas de Desempeño en Detección de Objetos . . . 36
3.6.1. Precision . . . 36
3.6.2. Sensibilidad (*Recall*) . . . 36
3.6.3. Intersection over Union (IoU) . . . 37
3.6.4. Cálculo de Intersección y Unión . . . 37
3.6.5. Importancia del IoU en la Evaluación de Modelos . . . 37
3.6.6. Impacto del IoU en la Detección de Objetos . . . 38
3.6.7. Average Precision (AP) . . . 38
3.6.8. Mean Average Precision (mAP) . . . 39

4. Resultados y Discusión **41**
4.1. Especificaciones del Hardware de Prueba . . . 43
4.2. Métricas de Evaluación . . . 43
4.3. Resultados de la Etapa de Validación . . . 44

5. Conclusiones **51**
5.1. Futuras líneas de Investigación . . . 53
5.1.1. Evaluación del Rendimiento en Diferentes Hardware . . . 53

Bibliografía **54**

Índice de Figuras

2.1. Ejemplo de identificación automática de personas y paquetes en cámaras de seguridad, basada en detección de objetos.(Ring, 2023) 17
2.2. Modelo implementado de YOLOv8 estandar (Terven et al., 2023). 21

4.1. Evolución de la métrica mAP por cada 10 épocas. (a) Modelo YOLOv8-Conv2D. (b) Modelo YOLOv8-Separable. 44
4.2. Modelo de detección con YOLOv8-Conv2D entrenado con 50 épocas. 46
4.3. Modelo de detección con YOLOv8-Separable entrenado con 100 épocas. 48

Índice de Tablas

2.1. Comparación de diferentes modelos de detección de objetos en términos de ventajas y limitaciones. 13

3.1. Parámetros del modelo implementado de YOLOv8. 35

4.1. Especificaciones del hardware de prueba. 43
4.2. Resultados en validación para ambos modelos. 45

Resumen

La detección de objetos ha tenido avances significativos y aplicaciones de uso común hasta el día de hoy. Una de las dificultades para su implementación eficiente radica en el hecho de que requiere un alto poder de cómputo, lo cual determina la calidad y rapidez en las detecciones.

Esta tesis evaluó dos modelos de detección de objetos YOLOv8, con el objetivo de analizar el compromiso entre desempeño en detección y eficiencia computacional en distintos entornos de ejecución. Se estudió una versión con redes neuronales con convoluciones estándar y una variante modificada con convoluciones separables en profundidad. Ambos modelos fueron implementados en un computador de escritorio y un dispositivo de capacidad limitada como Raspberry Pi.

La implementación se realizó en TensorFlow/Keras y la evaluación se efectuó sobre la base de datos de COCO2017, utilizando 118.287 imágenes para entrenamiento y 5.000 para validación. Se consideraron métricas estándar de detección y de eficiencia.

Los resultados indicaron que el modelo con convoluciones estándar obtuvo mejores indicadores de calidad en la detección de objetos que el modelo de convoluciones separables. La velocidad de detección en el modelo con convoluciones estándar, tanto para el procesamiento con CPU y con GPU resultó similar. En el caso de la implementación en Raspberry PI, el modelo con convoluciones separables obtuvo una mejor velocidad de detección, pero con menores indicadores de precisión.

Se concluyó que el desempeño del modelo con convoluciones estándar, utilizando infraestructura y configuraciones con mayor capacidad de cómputo, es superior principalmente en indicadores de calidad de detección. La velocidad de detección en los dispositivos resultó ser relativamente similar. Respecto al modelo de convoluciones separables, fue más adecuado en el dispositivo de capacidad reducida. Finalmente, la selección adecuada de la arquitectura del modelo de detección de objetos puede mejorar considerablemente la eficiencia de procesamiento sin comprometer en exceso la precisión.

Abstract

Object detection has experienced significant advances and has many common applications today. One of the main difficulties in its efficient implementation lies in the fact that it requires high computational power, which determines the quality and speed of detections.

This thesis evaluated two YOLOv8 object detection models, with the aim of analyzing the trade-off between detection performance and computational efficiency in different execution environments. A version with standard convolutional neural networks and a modified variant with separable convolutions in depth were studied. Both models were implemented on a desktop computer and a limited-capacity device such as Raspberry Pi.

The implementation was carried out using TensorFlow/Keras, and the evaluation was performed on the COCO2017 database, using 118,287 images for training and 5,000 for validation. Standard detection and efficiency metrics were considered.

The results indicated that the model with standard convolutions achieved better quality indicators in object detection than the separable convolutions model. The detection speed in the model with standard convolutions was similar for both CPU and GPU processing. In the case of the implementation on Raspberry Pi, the model with separable convolutions obtained a better detection speed, but with lower accuracy indicators.

It is concluded that the performance of the standard convolution model, using infrastructure and configurations with greater computing capacity, is superior mainly in detection quality indicators. The detection speed on the devices was relatively similar. The separable convolution model was more suitable for the device with reduced capacity. Finally, the appropriate selection of the object detection model architecture can significantly improve computational efficiency without excessively compromising accuracy.

Capítulo 1

Introducción

1.1. Descripción del Problema

La detección de objetos ha mejorado en los últimos años, impulsada por el desarrollo de modelos de redes neuronales convolucionales (CNN) más sofisticados y eficientes.

A pesar de estos avances, el problema central para su adopción en aplicaciones de la vida real no se limita sólo a la calidad de detección, sino a la viabilidad de ejecución en entornos con restricciones de cómputo.

En aplicaciones críticas como sistemas de asistencia vehicular, anti colisión, vigilancia y robótica, la detección debe ser precisa y oportuna para apoyar decisiones en tiempo real. Sin embargo, los modelos de Deep Learning empleados habitualmente requieren una capacidad de procesamiento elevada, lo que dificulta su despliegue en plataformas embebidas o dispositivos de bajo consumo (por ejemplo, Raspberry Pi), donde la memoria disponible, la potencia de CPU y el consumo energético son limitados.

En consecuencia, se genera un compromiso inevitable al intentar ejecutar modelos complejos en hardware reducido, la latencia aumenta y se compromete la operatividad del sistema, aun cuando el modelo mantenga un buen desempeño en infraestructura de alto rendimiento.

Este trabajo aborda este problema evaluando YOLOv8 en escenarios de capacidad de cómputo reducida. Específicamente, se comparó el modelo estándar con convoluciones tradicionales (Conv2D) frente a una versión modificada mediante la sustitución de estas capas por convoluciones separables en profundidad (SeparableConv2D), técnica que reduce el costo de procesamiento al descomponer la operación de convolución.

1.2. Justificación

La detección de objetos es un área central de la visión por computadora debido a su utilidad en múltiples aplicaciones, como seguridad, medicina, conducción autónoma y vigilancia. Dentro de este contexto, la familia de modelos YOLO (You Only Look Once) se ha consolidado por su capacidad para realizar detección en tiempo real con buen desempeño.

Este trabajo se enfocó específicamente en YOLOv8, considerando que ofrece un equilibrio favorable entre precisión, eficiencia, flexibilidad y facilidad de uso.

La justificación del proyecto se sustentó en tres aspectos relevantes.

Primero, la relevancia tecnológica de YOLOv8, al ser una arquitectura ampliamente utilizada.

Segundo, su amplio uso en entornos operativos reales, sustentado por la calidad y rapidez de detección son requerimientos clave en escenarios reales como vigilancia, reconocimiento facial y asistencia al conductor, donde evaluar el rendimiento del modelo es un paso necesario para su implementación.

Tercero, el manejo de recursos, ya que YOLOv8 está orientado a reducir el costo de cómputo en relación con otras alternativas de detección, lo que favorece su ejecución en sistemas con restricciones.

Además, se planteó modificar YOLOv8 mediante la incorporación de convoluciones separables en reemplazo de convoluciones 2D tradicionales, con el objetivo de analizar su impacto en precisión, velocidad de inferencia y eficiencia de cómputo.

Finalmente, el proyecto se justificó por la necesidad de determinar la viabilidad de despliegue en hardware reducido, comparando plataformas como Raspberry Pi y ESP32-CAM en función del procesamiento, la memoria y compatibilidad con modelos de detección.

1.3. Objetivos

1.3.1. Objetivo General

Analizar y evaluar el modelo YOLOv8 para la detección de objetos, implementándolo en un entorno de hardware con capacidades reducidas.

1.3.2. Objetivos Específicos

- Analizar la arquitectura y las mejoras en las técnicas introducidas en YOLOv8.
- Desarrollar versiones del modelo YOLOv8, en su configuración clásica y modificada, que permitan su análisis y evaluación experimental.
- Evaluar el rendimiento de los modelos en términos de precisión, velocidad y eficiencia computacional.
- Comparar el rendimiento de dos versiones de YOLOv8 en un hardware reducido.

1.4. Alcance del Proyecto

Esta tesis tuvo como alcance el estudio de la eficiencia de detección de objetos basados en modelos de YOlOv8, que incluyó un computador de escritorio y un dispositivo Rasperry PI y el análisis del desempeño de YOLOv8 con una implementación estándar y otra implementación mediante convoluciones separables. La evaluación se realizó utilizando la base de datos COCO2017 como referencia y así poder efectuar la experimentación de manera comparable y reproducible. La optimización y mejoras avanzadas del modelo, así como las implementaciones en otros entornos, quedaron fuera del alcance de este estudio.

1.5. Hipótesis de Investigación

Los modelos de Deep Learning adaptados para hardware reducido tendrán una precisión mayor que los modelos de Deep Learning tradicionales en función de los recursos disponibles, comparando métricas específicas de rendimiento.

Capítulo 2

Marco teórico

2.1. Historia y Evolución de la Detección de Objetos

La evolución de la detección de objetos, van desde métodos basados en características manuales hasta modelos de Deep Learning. En un comienzo, se utilizaban técnicas como SIFT (Scale-Invariant Feature Transform) y HOG (Histogram of Oriented Gradients) . Estas técnicas se basaban en la extracción de características específicas de las imágenes y su clasificación mediante algoritmos de aprendizaje automático tradicionales (Murel & Kavlakoglu, 2023).

La extracción manual de características, como SIFT y HOG, centradas en descriptores geométricos para capturar rasgos locales en las imágenes. Estos métodos fueron una base para la clasificación de objetos, pero sus limitaciones en flexibilidad y precisión impulsaron la introducción de las redes neuronales convolucionales (CNN).

Con la introducción de las Redes Neuronales Convolucionales (CNNs). AlexNet, presentada en 2012, demostró la eficiencia de las CNNs al ganar la competencia ImageNet. Este hito fue seguido por modelos como R-CNN (Regions with CNN), Fast R-CNN y Faster R-CNN, que mejoraron continuamente la precisión y la eficiencia al utilizar propuestas de regiones y técnicas de refinamiento de atributos (Liu et al., 2020).

En 2015, el modelo YOLO (You Only Look Once) mejoró el campo de la detección de objetos en tiempo real al permitir la identificación de múltiples clases con una sola pasada sobre la imagen. Se proporcionó un equilibrio entre precisión y eficiencia de procesamiento mediante una arquitectura completamente convolucional y la subdivisión de la imagen en una cuadrícula de regiones predictivas.

A diferencia de arquitecturas posteriores que incorporaron optimizaciones en las capas convolucionales, YOLO en su versión original emplea convoluciones estándar 2D para la extracción de características y la predicción de objetos en la imagen.

En versiones posteriores, se introdujeron mejoras como redes de pirámides de características (Feature Pyramid Networks, FPN) y estrategias avanzadas de procesamiento convolucional para mejorar la detección en múltiples escalas(Murel & Kavlakoglu, 2023).

La introducción de arquitecturas más avanzadas como SSD (Single Shot MultiBox Detector), FPN (Feature Pyramid Networks) y Transformers ha permitido mejorar aún más la precisión y la

velocidad de los modelos de detección de objetos. Estas innovaciones han sido impulsadas por el aumento en la capacidad de procesamiento de los GPUs y el desarrollo de nuevas técnicas de Deep Learning que permiten una mejor generalización y adaptación a diferentes entornos (Kaur & Singh, 2023; Liu et al., 2020; Zou et al., 2019).

2.2. Incremento en la Capacidad de Cómputo

El progreso de la detección de objetos ha dependido del avance del hardware que soporta el entrenamiento y la predicción de los modelos. En particular, el desarrollo de GPUs (Unidades de Procesamiento Gráfico) y TPUs (Tensor Processing Units) ha sido un catalizador para el uso de redes neuronales más profundas y complejas.

- **Evolución de GPUs y TPUs**: Las GPUs han desempeñado un papel fundamental en el avance del Deep Learning y la detección de objetos, proporcionando la capacidad de procesamiento paralelo necesaria para manejar grandes volúmenes de datos y entrenar redes neuronales profundas de manera eficiente. NVIDIA, por ejemplo, ha desarrollado GPUs específicamente para tareas de aprendizaje profundo, como la serie Tesla y A100, que ha permitido entrenar modelos complejos con una precisión sin precedentes.
- En 2016, Google introdujo las TPUs, diseñadas específicamente para acelerar el entrenamiento y la inferencia de modelos. Estas unidades de procesamiento, combinadas con arquitecturas de Deep Learning cada vez más avanzadas, han reducido drásticamente el tiempo requerido para entrenar modelos como YOLO o SSD, permitiendo una mejor adaptación en aplicaciones de tiempo real (Murel & Kavlakoglu, 2023).
- **Hardware especializado y eficiencia energética**: La implementación de modelos de detección de objetos en dispositivos móviles y sistemas embebidos enfrenta las limitaciones de recursos computacionales y consumo energético. Con el avance del *edge computing*, ha surgido la necesidad de hardware más eficiente que permiten ejecutar modelos avanzados sin comprometer la velocidad ni la precisión. En este contexto, dispositivos como la serie Jetson de NVIDIA y las Google Edge TPU han sido diseñados para ofrecer un alto rendimiento con un consumo energético optimizado. Estas plataformas permiten la predicción en tiempo real con modelos como YOLOv8, facilitando su aplicación en sistemas autónomos y de seguridad con restricciones de hardware.
- **Optimización mediante poda de parámetros y cuantización**: Con las mejoras en el hardware ha facilitado la adopción de técnicas de optimización como la poda de parámetros y la cuantización de pesos, reduciendo el tamaño y la carga de cálculo de los modelos sin afectar su precisión. La combinación de hardware especializado y estrategias de optimización ha permitido que modelos complejos sean ejecutados en dispositivos con capacidad limitada, como drones, cámaras de vigilancia y sistemas de automatización industrial, mejorando la eficiencia y reduciendo los costos operativos (Han et al., 2016).

Estos avances en hardware facilitaron el desarrollo de modelos de detección de objetos cada vez más sofisticados, haciendo posible su implementación en aplicaciones del mundo real que requieren de una alta precisión y tiempos de respuesta rápidos. Además, el uso de técnicas como redes neuronales ligeras (MobilNet, EfficientNet) que se implementaron a modelos en dispositivos con recursos limitados, mejoró así la eficiencia sin comprometer la calidad de los datos.

- **Convoluciones separables en profundidad**: Las convoluciones separables en profundidad (*Depthwise Separable Convolutions*) son una técnica de optimización ampliamente utilizada en modelos modernos de redes neuronales convolucionales, como MobileNet y Xception. Esta técnica consiste en dividir la operación de convolución tradicional en dos pasos: una convolución *depthwise*, que aplica un único filtro por canal de entrada, y una convolución *pointwise*, que combina las salidas anteriores mediante filtros 1x1 (Chollet, 2017; Howard et al., 2017).

  Este diseño permite reducir el número de parámetros y operaciones de procesamiento, manteniendo la capacidad de extracción de características. Gracias a esta eficiencia, su implementación se ha extendido en modelos diseñados para dispositivos móviles y tareas de inferencia en tiempo real, donde los recursos de cálculo son limitados (Howard et al., 2017).

  En YOLOv8, si bien su arquitectura base no incorpora directamente las convoluciones separables, algunos trabajos recientes han propuesto su integración como una variante experimental para mejorar el rendimiento en plataformas con capacidad reducida, mostrando resultados prometedores en términos de eficiencia sin sacrificar la precisión del modelo (Terven et al., 2023; Varghese & M., 2024).

Modelos	Ventajas	Limitaciones
SIFT y HOG (Zhao et al., 2019)	Efectivos en detección de objetos básicos en entornos con baja variabilidad, las características son consistentes.	Presentan dificultades frente a cambios de iluminación, rotaciones y escenarios dinámicos, además de depender de ingeniería manual de características.
AlexNet y CNNs (Kaur & Singh, 2023)	Permitió extracción automática de características sin necesidad de intervención manual a través de aprendizaje profundo, mejorando precisión y generalización.	Requieren grandes volúmenes de datos y alta capacidad de cómputo para entrenamiento eficiente, lo que aumenta los costos de implementación.
R-CNN (Kaur & Singh, 2023)	Integraron la extracción de regiones y clasificación; Faster R-CNN optimiza este proceso con redes de propuestas de regiones (RPN).	Complejidad en la composición del modelo que incrementa en el tiempo de entrenamiento demandando mayor carga de cálculo; requiere múltiples etapas de procesamiento.
YOLO (Kaur & Singh, 2023; Zhao et al., 2019; Zou et al., 2019)	Aplicó la detección en una sola pasada (single-shot), permitiendo la inferencia en tiempo real. Las convoluciones separables se ocupn principalmente en variantes livianas derivadas (YOLOv3-Lite/Micro-YOLO y adaptaciones ligeras basadas en YOLOv5).	Puede presentar dificultades en la detección precisa de objetos pequeños y en escenarios con múltiples clases y compromete ligeramente la precisión en favor de la velocidad.
SSD y FPN (Murel & Kavlakoglu, 2023)	SSD detecta múltiples escalas en una sola pasada, mientras que FPN emplea una arquitectura jerárquica de características para mejorar la detección de objetos de diversos tamaños.	Menor precisión comparando con modelos como YOLO o R-CNNs en ciertos escenarios; puede tener dificultades detectando los objetos pequeños.
Transformers (DETR) (Zou et al., 2019)	El uso Transformers en detección de objetos, mejoró las relaciones espaciales y contexto en imágenes complejas.	Modelos con alto costo de computación y alta densidad de datos para el entrenamiento; mayor dificultad de despliegue.

Tabla 2.1. Comparación de diferentes modelos de detección de objetos en términos de ventajas y limitaciones.

La Tabla 2.1 presenta una comparación entre algunos de los modelos más representativos, evaluando sus ventajas y limitaciones en términos de precisión, eficiencia de cómputo y aplicabilidad en distintos entornos.

2.3. Variantes de los Modelos de YOLO

Desde su introducción en 2015, los modelos de YOLO han conseguido la simplificación y aceleración de la detección de objetos. YOLO cambio las metodologías tradicionales al integrar la clasificación y localización en un solo paso, facilitando implementaciones en tiempo real mucho más rápidas que los enfoques convencionales (Murel & Kavlakoglu, 2023; Redmon et al., 2016).

2.4. YOLOv1 a YOLOv3

Los primeros modelos de YOLO se centraron en mejorar la rapidez y precisión, introduciendo técnicas innovadoras para las predicciones de cajas delimitadoras y probabilidades de clases directamente desde las características visuales completas.

- YOLOv1: Presentado en 2015, revolucionó el campo al permitir la detección en tiempo real. Dividía la imagen en una cuadrícula y predecía cajas y probabilidades para cada celda, lo que simplificaba enormemente el proceso de detección (Redmon et al., 2016).
- YOLOv2 (YOLO9000): Introdujo mejoras significativas, como la normalización por lotes y una red de características mejorada (Darknet-19), lo que incrementó la precisión y permitió la detección de más de 9000 clases de objetos mediante la combinación de datos supervisados y no supervisados (Redmon & Farhadi, 2017).
- YOLOv3: Comenzó a utilizar una red más profunda (Darknet-53) y predicciones en tres escalas diferentes. Esto mejoró la detección de objetos pequeños y permitió una mayor flexibilidad en la predicción de objetos de diferentes tamaños (Redmon & Farhadi, 2017).

2.5. YOLOv4 a YOLOv5

Estas versiones introdujeron mejoras en la gestión de escalas de objetos y eficiencia en la utilización de GPU, permitiendo un rendimiento robusto incluso en hardware menos potente.

- YOLOv4: Lanzado en 2020, mejoró tanto la velocidad como la precisión mediante técnicas avanzadas como Mosaic (una técnica de aumento de datos) y Self-Adversarial Training (SAT), que mejoran la capacidad del modelo para generalizar a nuevos datos(Bochkovskiy et al., 2020).
- YOLOv5: A pesar de que el desarrollo fue de manera independiente y con algunas controversias sobre su continuidad directa con los modelos anteriores, YOLOv5 se centró en la accesibilidad

y eficiencia de implementación. Incorporó una arquitectura eficiente y diversas mejoras, facilitando su despliegue en diferentes plataformas de hardware, incluyendo GPUs (Jocher et al., 2022).

2.6. YOLOv6 y YOLOv7

Estas iteraciones avanzaron en la precisión mediante técnicas de *Deep Learning* más sofisticadas y ajustes en la arquitectura de red para manejar una escala más amplia de tamaños y formas de objetos.

- **YOLOv6**: Introdujo una arquitectura de red orientada a escenarios industriales, optimizada para mejorar la eficiencia de cálculo y la capacidad de detección. Sus principales aportes se centran en el diseño de bloques convolucionales optimizados y en el uso de técnicas de re-parametrización estructural, que permiten entrenar bloques multi-rama y fusionarlos en una única convolución estándar en la fase de predicción. A diferencia de las convoluciones separables, que descomponen la operación en una convolución depthwise seguida de una convolución pointwise para reducir el número de parámetros, la re-parametrización de YOLOv6 mantiene convoluciones densas equivalentes, pero simplifica su estructura para hacer más eficiente el despliegue, además de mejorar la estabilidad del entrenamiento, sin emplear convoluciones separables como componente central de la arquitectura (Meituan, 2022).
- **YOLOv7**: En su versión original, utiliza convoluciones 2D estándar. Sin embargo, investigaciones posteriores han propuesto modificaciones en la arquitectura que incorporan convoluciones separables en profundidad, especialmente en los componentes MP1 (Max Pooling layer, utilizado para la reducción espacial de las características) y ELAN (Efficient Layer Aggregation Network, un módulo diseñado para mejorar la agregación y propagación de características en el backbone del modelo), lo que ha demostrado mejorar la eficiencia de cálculo sin degradar el rendimiento. (Zhang & Wang, 2024).

2.7. Aplicaciones de Modelos de Detección de Objetos

Los modelos de detección de objetos, como YOLO, han sido ampliamente adoptados en diversas industrias debido a su capacidad para identificar y localizar objetos en imágenes y videos en tiempo real. Esta capacidad es fundamental en aplicaciones que requieren toma de decisiones rápida y precisa. A continuación, se detallan algunas de las aplicaciones más relevantes con un enfoque técnico y funcional basado en la información analizada en este estudio.

2.7.1. Vehículos Autónomos

La percepción del entorno es un componente crítico en los sistemas de conducción autónoma. Los modelos de detección de objetos permiten a los vehículos identificar y clasificar en tiempo real

elementos clave en la vía, como otros vehículos, peatones, señales de tráfico y obstáculos. YOLO es particularmente útil en esta aplicación debido a su arquitectura optimizada para el procesamiento en tiempo real con una alta tasa de detección.

Tesla y Waymo han desarrollado sistemas avanzados de percepción basados en modelos de detección de objetos. Tesla, a través de su sistema Autopilot, emplea una red neuronal convolucional de múltiples capas que integra YOLO y otras arquitecturas similares para detectar objetos en diferentes condiciones climáticas y de iluminación. Su sistema opera en conjunto con sensores como cámaras RGB, radares y LiDAR para mejorar la detección y reducir los falsos positivos en entornos urbanos y carreteras de alta velocidad. La integración de YOLO permite el reconocimiento instantáneo de objetos en escenas complejas con una latencia mínima, optimizando la capacidad del vehículo para realizar maniobras seguras (Ultralytics, 2024; V7 Labs, 2024).

Por otro lado, Waymo ha desarrollado un modelo híbrido que combina detección de objetos basada en YOLO con segmentación semántica para mejorar la identificación de elementos estáticos y en movimiento. Sus algoritmos permiten el mapeo detallado del entorno y la predicción de trayectorias de otros vehículos y peatones, lo que aumenta la seguridad y la eficiencia del sistema de navegación autónoma.

2.7.2. Seguridad y Vigilancia

En aplicaciones de seguridad, los modelos de detección de objetos han sido implementados en sistemas de video vigilancia inteligente para identificar intrusos, detectar comportamientos sospechosos y generar alertas en tiempo real. YOLO destaca en este ámbito debido a su capacidad de detección con alta precisión y bajo consumo de procesamiento, lo que permite su ejecución en dispositivos reducidos, como cámaras de seguridad con procesamiento en el dispositivo. Además, su arquitectura optimizada para la detección rápida permite el análisis en tiempo real de múltiples flujos de video simultáneamente. A continuación, algunos casos de uso:

- **Cámaras de seguridad inteligentes (Ring - Amazon):** Ring, la división de seguridad de Amazon, ha integrado modelos de detección de objetos en sus cámaras de seguridad para mejorar la identificación de personas y paquetes. A través de YOLO, las cámaras pueden distinguir entre objetos en movimiento y personas, activando notificaciones automáticas para los propietarios en función del contexto detectado. La optimización de YOLO para dispositivos con restricciones de hardware ha permitido que estos sistemas funcionen de manera eficiente en cámaras alimentadas por batería, reduciendo el consumo energético sin comprometer la precisión en la detección de eventos. Adicionalmente, estos dispositivos utilizan técnicas para la reducción de falsos positivos basadas en análisis de patrones de movimiento y reconocimiento de características específicas, como se muestra en la Figura 2.1, las cámaras Ring permiten la identificación automática de personas y paquetes. (Ring, 2023).

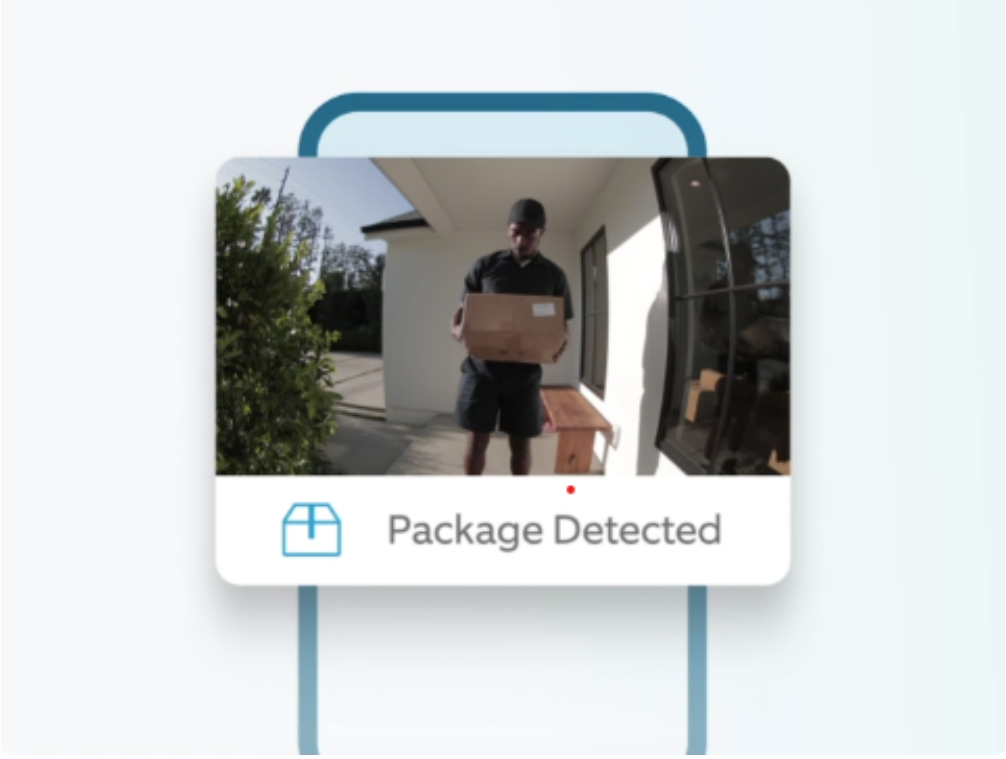


Figura 2.1. Ejemplo de identificación automática de personas y paquetes en cámaras de seguridad, basada en detección de objetos.(Ring, 2023)

- **Aeropuertos y monitoreo de accesos (Aeropuerto de Changai - Singapur):** En el Aeropuerto de Changai, en Singapur, ha implementado un sistema de monitoreo basado en inteligencia artificial que utiliza YOLO para la detección de objetos sospechosos en áreas de acceso restringido. Este sistema no solo rastrea a individuos en tiempo real, sino que también analiza el comportamiento de grupos de personas para detectar patrones inusuales, como reuniones prolongadas en lugares inusuales o el abandono de equipaje. La integración de YOLO con redes neuronales recurrentes permite predecir comportamientos potencialmente riesgosos mediante el análisis de movimientos y trayectorias. Para mitigar falsos positivos, el sistema emplea modelos adicionales de clasificación que descartan alertas por movimientos no sospechosos (Changi Airport Group, 2023).
- **Seguridad en eventos masivos (Super Bowl LVII - EE.UU.):** Durante el Super Bowl LVII, las autoridades utilizaron un sistema de video vigilancia con inteligencia artificial basado en YOLO para detectar comportamientos sospechosos en la multitud. Este sistema permitió una detección temprana de posibles amenazas, como individuos con actitudes agresivas o movimientos anómalos en áreas sensibles. Gracias a la capacidad de YOLO de procesar múltiples cuadros por segundo en tiempo real, las fuerzas de seguridad pudieron identificar y actuar preventivamente ante posibles situaciones de riesgo antes de que ocurrieran incidentes. Para mejorar la eficiencia del sistema, se integraron técnicas de filtrado de detecciones basadas en

análisis contextual, reduciendo así el número de alertas erróneas y mejorando la precisión de las intervenciones (NFL Security Team, 2023).

2.7.3. Industria y Manufactura

En la industria manufacturera, la detección de objetos ha mejorado los procesos de control de calidad e inspección automatizada. Los modelos de detección permiten identificar defectos en productos, monitorizar el ensamblaje y garantizar que los estándares de calidad se cumplan en tiempo real. YOLO es una opción ideal debido a su capacidad de procesar imágenes a alta velocidad sin requerir hardware especializado de alto costo. Además, su compatibilidad con arquitecturas optimizadas permite la integración con sistemas industriales como SCADA, PLCs y robots de ensamblaje. Algunos casos de uso relevantes a mencionar:

- **Inspección de ensamblaje y calidad en la industria automotriz (BMW - Alemania)**: En la planta de producción de BMW en Múnich se han implementado sistemas de visión artificial basados en YOLO para la inspección de carrocerías. Estos sistemas verifican la alineación de paneles, la instalación de piezas y la ausencia de defectos estéticos en tiempo real. A través de un conjunto de cámaras de alta resolución, YOLO procesa imágenes de diferentes ángulos de los vehículos en ensamblaje, identificando anomalías con un margen de error inferior al 2 %. El uso de esta tecnología ha permitido reducir los tiempos de inspección en un 40 % en comparación con métodos tradicionales de control de calidad, además de minimizar el número de defectos en los vehículos entregados a los clientes. Para optimizar la detección en entornos industriales con variaciones de iluminación, se han implementado modelos de YOLO entrenados con técnicas de data augmentation y normalización de brillo (BMW Manufacturing, 2023).

- **Fabricación de componentes electrónicos (Foxconn - China)**: Foxconn, uno de los mayores fabricantes de dispositivos electrónicos del mundo, utiliza YOLO en sus líneas de producción para identificar fallos en circuitos y ensamblajes de chips. La implementación de YOLO permite detectar defectos microscópicos en placas base y pantallas OLED mediante el uso de cámaras de inspección óptica. La integración con brazos robóticos ha optimizado la velocidad del proceso, reduciendo los rechazos en un 30 % y mejorando la eficiencia de producción al disminuir la necesidad de inspección manual. Además, los modelos de YOLO han sido optimizados con técnicas de cuantización para operar eficientemente en hardware industrial con recursos limitados. Se han incorporado filtros basados en inteligencia artificial para reducir la cantidad de falsos positivos en la inspección, asegurando una mayor precisión en el control de calidad (Foxconn Technology Group, 2023).

- **Automatización en la industria química (BASF - Alemania)**: BASF ha integrado modelos de detección de objetos basados en YOLO en sus plantas de producción para automatizar la clasificación de productos químicos según su etiquetado y nivel de peligrosidad. Los sistemas identifican envases con errores en el etiquetado y activan procesos automáticos de re etiquetado, minimizando el riesgo de distribución incorrecta de productos químicos peligrosos. Mediante el

uso de modelos entrenados con imágenes de miles de productos químicos, YOLO ha permitido reducir los errores de clasificación en un 25 %, mejorando la seguridad en la manipulación y distribución de sustancias peligrosas. La integración con sistemas de visión espectral ha permitido mejorar la detección de etiquetas dañadas o parcialmente visibles, asegurando una mayor confiabilidad en el proceso de clasificación (BASF Chemicals, 2023).

La implementación de modelos de detección de objetos ha demostrado ser una solución eficiente para mejorar los procesos en diversas industrias. Su capacidad para detectar objetos en tiempo real con alta precisión lo convierte en una herramienta fundamental en la evolución de la inteligencia artificial aplicada. En el futuro, se espera que estos modelos sigan mejorando en eficiencia de cómputo y precisión, facilitando su implementación en dispositivos con restricciones de hardware y expandiendo su aplicabilidad a nuevas áreas tecnológicas.

2.8. Introducción al Modelo YOLOv8

La versión YOLOv8, lanzada en 2023, incorpora algunas innovaciones en detección de objetos, destacando por su velocidad mejorada y precisión en la detección. Utiliza técnicas de optimización como ajustes dinámicos durante el entrenamiento y un mejor manejo de variaciones de imagen, resultando en una precisión superior y una velocidad impresionante, incluso en plataformas computacionales limitadas (Varghese & M., 2024).

YOLOv8 ofrece variantes ligeras y variantes de mayor capacidad, lo que permite ajustar el número de parámetros y la potencia de representación de acuerdo con las restricciones del hardware disponible. Esta flexibilidad resulta especialmente relevante en escenarios donde se requiere ejecutar la detección de objetos en tiempo real sobre dispositivos con recursos de cómputo acotados, tales como sistemas embebidos o computadoras de placa reducida. El análisis de versiones compactas de YOLOv8 y de técnicas orientadas a optimizar sus bloques convolucionales cobra particular importancia, ya que permite estudiar el compromiso entre precisión, velocidad de detección y uso de memoria, eje central del presente trabajo.

2.8.1. Impacto y Aplicaciones de YOLOv8

YOLOv8 ha ampliado su impacto más allá del ámbito académico, consolidándose en sectores críticos como la seguridad pública y la industria automotriz, donde contribuye a mejorar la eficiencia y la confiabilidad de sistemas de vigilancia y de vehículos autónomos. Su capacidad de predicción en tiempo real, junto con un desempeño competitivo en precisión, lo convierte en una alternativa relevante para aplicaciones que requieren respuestas rápidas y consistentes (Varghese & M., 2024).

Respecto al modelo YOLOv8 empleado en este trabajo, la Figura 2.2 presenta la arquitectura base de YOLOv8 empleada en este trabajo, destacando la estructura de sus componentes principales (Backbone, Neck y Head). La figura resume el flujo general del modelo: el Backbone extrae características jerárquicas desde la imagen de entrada; el Neck integra información multiescala mediante operaciones de upsample y concat para fusionar representaciones; y el Head genera predicciones en distintas resoluciones, estimando coordenadas de cajas y probabilidades de clase para objetos de diferentes tamaños.

A continuación, se describen las partes que conforman la arquitectura general del modelo:

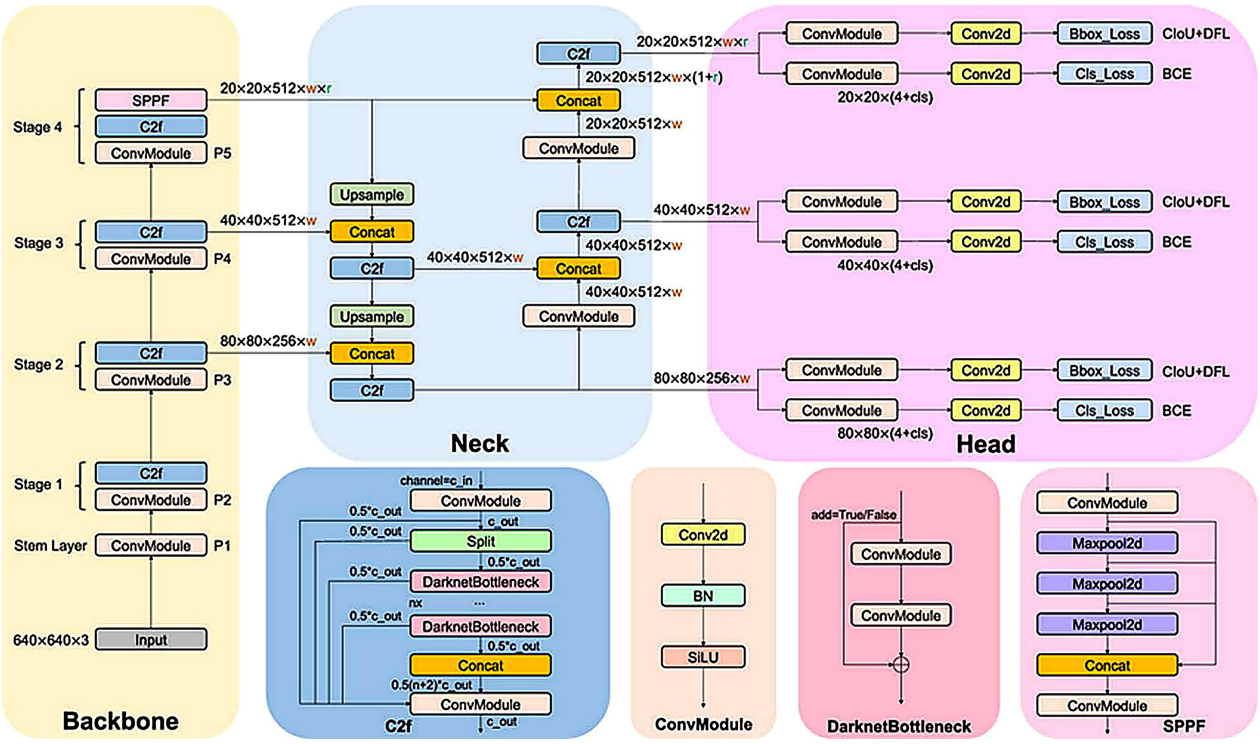


Figura 2.2. Modelo implementado de YOLOv8 estandar (Terven et al., 2023).

2.8.2. Backbone

Tiene como función ser el componente encargado de extraer características básicas de la imagen. Procesa la imagen de entrada y produce mapas de características de alto nivel y está compuesto por:

- **Arquitectura**: En YOLOv8, el Backbone se basa en CSPDarknet, una variante del Darknet utilizada en versiones anteriores de YOLO. CSPDarknet mejora el aprendizaje de características y la eficiencia dividiendo la red en dos partes (Wang et al., 2019).
- **Capas convolucionales (Conv2D)**: Estas capas aplican filtros a las imágenes de entrada para extraer características importantes. Los filtros son pequeñas matrices que se mueven sobre la imagen, detectando bordes, texturas y patrones (Lecun et al., 1998).

- **Batch Normalization (Normalización por lotes)**: Esta técnica normaliza la salida de una capa previa, mejorando la estabilidad del modelo (Ioffe & Szegedy, 2015).

2.8.3. Funciones de Activación en YOLOv8

En YOLOv8, la función de activación utilizada es SiLU (Swish Linear Unit), la cual ha demostrado ser más eficiente en la propagación de gradientes y en la estabilización del entrenamiento de redes profundas. SiLU se define matemáticamente como:

$$SiLU(x) = x \cdot \sigma(x) \tag{2.1}$$

donde $\sigma(x)$ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2.2}$$

El principal beneficio de SiLU sobre otras funciones de activación como ReLU o Leaky ReLU radica en su suavidad y capacidad para controlar mejor los valores negativos del gradiente, lo que resulta en:

- **Mejor flujo de gradientes:** Reduce el problema de desaparición del gradiente (vanishing gradient) en redes profundas, facilitando un aprendizaje más estable.
- **Mayor expresividad del modelo:** A diferencia de ReLU, SiLU conserva valores negativos útiles en la red neuronal, permitiendo que el modelo aprenda relaciones más complejas.
- **Mejor rendimiento en tareas de visión:** Experimentos han demostrado que SiLU mejora la precisión de detección y segmentación en modelos como YOLOv8.
- **Continuidad y suavidad:** No introduce puntos de discontinuidad como ReLU, lo que mejora la estabilidad en el entrenamiento.

En YOLOv8, el uso de SiLU contribuye a un mejor ajuste del modelo y una mayor precisión en la detección de objetos, particularmente en escenarios con alta variabilidad de iluminación y texturas (Ultralytics, 2024).

2.8.4. Neck

Neck es una de las partes fundamentales dentro de la arquitectura de los modelos de detección de objetos, ya que actúa como un puente entre el *Backbone* y el *Head*. Su función principal es consolidar y refinar las características extraídas en las etapas previas, mejorando la representación de la información relevante para la detección de objetos.

En modelos de detección como YOLOv8, el *Neck* mejora la capacidad del modelo para manejar escalas y variaciones en la apariencia de los objetos. Esto es crucial en aplicaciones donde los objetos pueden estar distribuidos en diferentes tamaños dentro de la imagen de entrada. Además, este

módulo contribuye a la mejora de la capacidad de generalización del modelo, permitiéndole capturar características discriminativas de manera eficiente.

El diseño del *Neck* ha incorporado estrategias que favorecen la alta resolución, junto con descriptores de niveles más profundos de la red. Se emplean técnicas que combinan características de múltiples escalas para mejorar la capacidad de detección en distintos niveles de detalle. Esto permite al modelo identificar tanto objetos pequeños como grandes dentro de una misma imagen, lo que resulta esencial en escenarios del mundo real.

Para lograr estos objetivos, *Neck* emplea componentes clave:

- **Fusión de características**: Utiliza estructuras basadas en pirámide de características (*Feature Pyramid Network, FPN*) y la red de agregación de rutas (*Path Aggregation Network, PANet*) para combinar información de múltiples escalas. Estas técnicas permiten al modelo aprovechar tanto las características de bajo nivel como las de alto nivel para mejorar los detalles de la precisión de detección (He et al., 2015; Liu et al., 2020; Long et al., 2015).
- **Refinamiento de características**: Las características extraídas en el *Backbone* suelen contener información irrelevante. El *Neck* refina esta información a través de capas especializadas, asegurando que las representaciones transmitidas al *Head* sean más discriminativas y útiles para la detección de objetos.
- **Capas de concatenación (*Concatenate*)**: Estas capas combinan características de diferentes niveles de la red neuronal, integrando información de distintas escalas y mejorando la capacidad del modelo para detectar objetos de diferentes tamaños (Long et al., 2015).
- **Upsampling (Sobremuestreo inverso)**: Se utilizan técnicas de sobre muestreo para reconstruir información espacial perdida durante las operaciones de convolución y agrupamiento (*pooling*). Esto permite al modelo mejorar la localización de los objetos en la imagen. En YOLOv8, el *upsampling* se implementa mediante operaciones de interpolación y transposed convolutions (Odena et al., 2016).
- **Eficiencia computacional**: En YOLOv8, el *Neck* se ha diseñado para reducir el costo de procesamiento sin afectar la precisión. Aunque la arquitectura original no incorpora convoluciones separables de forma nativa, estas pueden integrarse manualmente como parte de adaptaciones personalizadas orientadas a plataformas de bajo consumo. La aplicación de convoluciones separables en profundidad, inspirada en arquitecturas eficientes como MobileNet y MobileNetV2, permite disminuir la cantidad de operaciones requeridas, favoreciendo un procesamiento más ágil sin sacrificar el rendimiento en tareas de detección (Howard et al., 2017; Sandler et al., 2018).

El *Neck* también desempeña una en la mejora de la robustez del modelo, ayudando a minimizar errores de detección y a mejorar la capacidad del sistema para adaptarse a diferentes condiciones de iluminación, escalas y orientaciones. Su correcta implementación es crucial para lograr un balance entre precisión y rendimiento de cálculo en tareas de detección de objetos en tiempo real.

2.8.5. Head

El Head del modelo YOLOv8 es el componente encargado de generar las predicciones finales de detección de objetos, incluyendo las clases y las coordenadas de las cajas delimitadoras. Su arquitectura está diseñada para mejorar la precisión y la eficiencia de cómputo, incorporando capas convolucionales especializadas y procesos de post procesamiento para mejorar la calidad de las predicciones.

- **Capas Convolucionales** :

  En la arquitectura del *Head* de YOLOv8 está diseñada para realizar las predicciones finales de las cajas delimitadoras, clases y puntuaciones de confianza, a partir de las características procesadas por el textitNeck. Este módulo emplea capas convolucionales estándar (`Conv2D`) con activaciones SiLU, manteniendo una estructura ligera pero precisa para detección en múltiples escalas. En su diseño convencional, YOLOv8 utiliza convoluciones de tipo estándar que aplican filtros en todas las dimensiones de la entrada, lo que, si bien garantiza una alta precisión, conlleva un mayor costo de procesamiento en comparación con versiones optimizadas.

- **Generación de Predicciones a Múltiples Escalas**:

  Para abordar la detección de objetos de diferentes tamaños dentro de una misma imagen, el *Head* de YOLOv8 implementa una estrategia de predicción en tres niveles jerárquicos (Bochkovskiy et al., 2020; Wang et al., 2019). Las cuales son:

  - `yolo_v8_head_1`: Especializado en la detección de objetos de menor tamaño.
  - `yolo_v8_head_2`: Diseñado para la detección de objetos de tamaño intermedio.
  - `yolo_v8_head_3`: Optimizado para la detección de objetos de gran tamaño.

  Cada uno de estos niveles recibe características refinadas del *Neck* y genera predicciones adaptadas a la escala de los objetos detectados, asegurando una mejor cobertura y precisión en escenarios con variabilidad en las dimensiones de los elementos presentes en la imagen.

- **Mapas de Predicción**: Dentro del *Head*, se generan dos tipos de mapas de predicción en cada una de las escalas mencionadas (Bochkovskiy et al., 2020; Lin et al., 2014):

  - **Mapa de clasificación**: Encargado de estimar la probabilidad de cada clase en cada celda de la cuadrícula de salida.
  - **Mapa de regresión de cajas delimitadoras**: Responsable de predecir la ubicación precisa de los objetos detectados mediante la estimación de coordenadas normalizadas en el formato (x, y, w, h), donde x e y representan el centro de la caja, y w y h definen su ancho y altura. Cada celda de la cuadrícula de predicción puede generar múltiples propuestas de cajas delimitadoras, cada una con una puntuación de confianza asociada.

- **Supresión de No Máximos (Non-Maximum Suppression, NMS)**:

  Dado que el modelo puede generar múltiples predicciones para un mismo objeto, se realiza un proceso de supresión de no máximos (*Non-Maximum Suppression, NMS*) para filtrar aquellas detecciones redundantes y seleccionar las más relevantes (Neubeck & Van Gool, 2006; Redmon & Farhadi, 2017).

  Este proceso se basa en los siguientes criterios:

- Se conservan las predicciones con la mayor confianza de detección.

- Se eliminan las detecciones cuyo solapamiento (*Intersection over Union, IoU*) con una predicción de mayor confianza supere un umbral predefinido.

  La aplicación de NMS contribuye a mejorar la precisión general del modelo y a evitar falsos positivos, asegurando que solo se mantengan las detecciones más relevantes en la salida final (Terven et al., 2023).

- **Post procesamiento y generación de la salida final**:

  Tras la aplicación de NMS, el modelo produce la salida final que incluye :

  - Coordenadas ajustadas de la caja delimitadora;
  - Puntuaciones de confianza asociadas a cada clase detectada;
  - Etiquetas de las clases con mayor probabilidad.

  Esta información se organiza en un formato estructurado que facilita su interpretación y permite su uso en aplicaciones prácticas de detección de objetos en tiempo real.

2.8.6. Componentes Avanzados

- **Residual blocks**: Son un componente clave en los modelos modernos de aprendizaje profundo, ya que ayudan a abordar el problema del gradiente que se desvanece, permitiendo el entrenamiento de redes neuronales muy profundas mediante la incorporación de conexiones de residuales (*Residual connections*) (He et al., 2016).

- **Un bloque residual básico tiene la siguiente arquitectura**:

  - **Capas convolucionales**: Usualmente, son dos capas de convolución con tamaño de kernel de 3x3 y *padding* de 1.
  - **Normalización de lotes (Batch Normalization)**: Aplicada después de cada capa de convolución para estabilizar y mejorar el proceso de entrenamiento (Ioffe & Szegedy, 2015).
  - **Funciones de activación no lineales**: En YOLOv8, se emplea la función de activación **SiLU** , debido a su capacidad para mejorar la propagación del gradiente y proporcionar una mayor suavidad en la transformación de características (Hendrycks & Gimpel, 2023).

- **Conexión residual (Residual Connections)**: En la arquitectura de YOLOv8, las conexiones residuales desempeñan un rol esencial en la preservación de la información a lo largo de la red, particularmente en las capas más profundas. Este mecanismo consiste en realizar una suma elemento a elemento (*element-wise addition*) entre la entrada original de un bloque y la salida de una capa posterior dentro del mismo, permitiendo así que los gradientes fluyan de forma más eficiente durante el proceso de retropropagación.

  Esta técnica fue introducida originalmente en las redes ResNet para mitigar el problema del desvanecimiento del gradiente en arquitecturas profundas (He et al., 2016), y ha sido adoptada en modelos modernos como YOLOv8 debido a su efectividad para mantener la estabilidad del entrenamiento y mejorar la convergencia. En el caso específico de YOLOv8, estas conexiones permiten conservar características semánticas clave desde las etapas tempranas de procesamiento, facilitando una detección más precisa al combinar información de diferentes niveles de abstracción (Varghese & M., 2024).

  También, su implementación no incrementa la complejidad de cálculo, lo que las hace especialmente adecuadas para modelos optimizados para tareas en tiempo real como la detección de objetos.

- **C2f (Cross Stage Partial Networks)**: El módulo C2f mejora la capacidad de aprendizaje del modelo al introducir conexiones parciales entre etapas, facilitando una mejor propagación de la información (Wang et al., 2019).

  - **Conexiones de atajo**: Las conexiones de atajo (*shortcuts*) permiten que la información fluya más libremente entre capas, mejorando la rapidez del aprendizaje y la capacidad del modelo para captar características complejas.
  - **Convolucionales y Normalización**: Utiliza convoluciones 1x1 y 3x3 junto con la normalización por lotes para procesar y combinar características de entrada y de salida de manera eficiente.

- **ConvModule**: Son bloques básicos de construcción presentes en la arquitectura de YOLOv8, encargados de aplicar secuencias de operaciones fundamentales sobre los mapas de características. Cada módulo incluye típicamente una convolución 1×1 o 3×3, seguida de una operación de normalización por lotes (*Batch Normalization*) y una función de activación no lineal, usualmente **SiLU** (Ioffe & Szegedy, 2015).

  Este diseño modular permite una mayor estabilidad durante el entrenamiento y mejora la expresividad del modelo sin aumentar la complejidad de cómputo. Además, su implementación estandarizada facilita la reutilización de bloques a lo largo de la red, lo que contribuye a una mayor rapidez estructural y consistencia en el flujo de información (Ultralytics, 2024; Varghese & M., 2024).

- **DarknetBottleneck**: Es una variante del bloque residual que facilita el entrenamiento de redes profundas al incluir capas convolucionales y conexiones de salto (He et al., 2016).

- **Bloques Residuales**: Similar a los bloques residuales en ResNet, pero adaptados específicamente para la arquitectura Darknet. Estos bloques permiten un flujo de gradiente más efectivo durante el entrenamiento, ayudando a mitigar problemas de degradación del gradiente.

- **SPPF**: El módulo SPPF (Spatial Pyramid Pooling – Fast) permite al modelo capturar información multiescala de forma eficiente al combinar atributos de distintos niveles de granularidad (Bochkovskiy et al., 2020). Para ello, emplea un esquema de pooling espacial piramidal que utiliza diferentes tamaños de ventana de pooling, integrando simultáneamente detalles finos y dependencias de gran escala. En conjunto, esta estrategia mantiene un bajo costo de procesamiento y contribuye a reducir la pérdida de información espacial relevante durante el procesamiento.

2.9. Convolución Separable

La convolución separable en profundidad (Depthwise Separable Convolution) se implementó con el objetivo de reducir la exigencia de parámetros. Esto se logra separando el filtrado espacial y la combinación de canales en dos etapas independientes:

2.9.1. Convolución Depthwise

La convolución depthwise (*Depthwise Convolution*) es una variante optimizada de la operación convolucional tradicional utilizada en redes neuronales profundas. A diferencia de una convolución estándar, donde cada filtro opera en todas las dimensiones del volumen de entrada (altura, ancho y profundidad de canales), la convolución depthwise aplica una convolución independiente por cada canal de entrada, reduciendo la cantidad de parámetros y las operaciones de cómputo involucradas.

Matemáticamente, en una convolución estándar, se utiliza un kernel de tamaño $k \times k$ con C_{in} canales de entrada, generando una salida con C_{out} canales. El número total de parámetros en esta operación es:

$$P_{\text{conv estándar}} = k \times k \times C_{\text{in}} \times C_{\text{out}} \tag{2.3}$$

donde:

- k es el tamaño del kernel (ej. 3×3 o 5×5), que define el área espacial sobre la cual se aplican los filtros en cada paso de la convolución.

- C_{in} es el número de canales de entrada, es decir, la profundidad de la imagen de entrada (ejemplo: $C_{\text{in}} = 3$ para una imagen RGB).

- C_{out} es el número de filtros utilizados, lo que determina la cantidad de canales en la salida de la convolución.

No obstante, la convolución depthwise emplea un filtro separado para cada canal de entrada, lo que reduce los cálculos a:

$$P_{\text{depthwise}} = k \times k \times C_{\text{in}} \tag{2.4}$$

Aquí se observa que el número de parámetros se reduce en un factor de $1/C_{\text{out}}$ en comparación con una convolución estándar. Esta reducción es clave para mejorar la rapidez de procesamiento, ya que disminuye la carga de cómputo y memoria, lo que la hace ideal para aplicaciones en dispositivos con restricciones de hardware, como teléfonos móviles o sistemas embebidos.

De esta forma, permite mantener la extracción de características espaciales dentro de cada canal individualmente, sin combinar la información entre los distintos canales de entrada. No obstante, debido a esta independencia entre canales, la convolución depthwise por sí sola no puede aprender interacciones entre características de diferentes canales, lo que limita su capacidad de representación en modelos más complejos (Howard et al., 2017).

2.9.2. Convolución Pointwise

Para superar la limitación de la convolución depthwise en la captura de interacciones entre canales, se introduce la convolución pointwise (*Pointwise Convolution*). Esta operación consiste en aplicar una convolución 1×1 sobre la salida de la convolución depthwise, permitiendo la combinación de información entre canales de manera eficiente.

Matemáticamente, la convolución pointwise aplica C_{out} filtros de tamaño 1×1, lo que genera:

$$P_{\text{pointwise}} = C_{\text{in}} \times C_{\text{out}} \tag{2.5}$$

donde:

- 1×1 representa el tamaño del kernel, que cubre todo el canal de entrada sin afectar la resolución espacial de la imagen.
- C_{in} es el número de canales de entrada provenientes de la convolución depthwise.
- C_{out} es el número de filtros, que define cuántos canales de salida tendrá la imagen resultante.

Esta operación funciona como una proyección lineal que conserva la resolución espacial de los mapas de características y permite combinar información de diferentes canales. La combinación de convoluciones depthwise y pointwise es fundamental en arquitecturas optimizadas para rendimiento computacional, como *MobileNet* y *Xception*, debido a su capacidad de reducir la cantidad de operaciones sin sacrificar precisión (Chollet, 2017; Howard et al., 2017).

En investigaciones recientes, algunos trabajos han explorado la incorporación experimental de convoluciones separables en arquitecturas YOLO modernas, proponiendo variantes no oficiales con el objetivo de mejorar la optimización de recursos en dispositivos con recursos limitados (Terven et al., 2023; Varghese & M., 2024).

2.9.3. Ventajas de las Convoluciones Separables

- **Reducción en el número de parámetros:** Las convoluciones estándar aplican un filtro 3D a los datos de entrada, lo cual puede ser computacionalmente costoso. Por otro lado, las convoluciones depthwise aplican filtros 2D separados a cada canal de entrada, lo que reduce el número de operaciones(Chollet, 2017). La convolucional pointwise (1x1) combina los resultados de las convoluciones depthwise, manteniendo la capacidad de captura de características, pero con menos parámetros y operaciones en comparación con una convolución estándar (Lin et al., 2014).

- **Eficiencia computacional:** Menos parámetros significan menos operaciones matemáticas durante el entrenamiento y la inferencia, lo cual resulta en una ejecución más rápida y eficiente en términos de memoria y potencia de procesamiento (Chollet, 2017). Esto es especialmente importante en aplicaciones donde el tiempo de respuesta y el uso eficiente de recursos son críticos.

- **Mejora del rendimiento en dispositivos móviles:** Las convoluciones separables son particularmente beneficiosas en dispositivos con recursos limitados, como una ESP32CAM, en donde la rapidez de cálculo y el uso reducido de memoria son críticos (Sandler et al., 2018). Al reducir la complejidad del modelo, es posible ejecutar modelos de redes neuronales más complejos en dispositivos con capacidad de procesamiento limitada.

2.10. Comparativa entre Raspberry Pi y ESP32-CAM

La selección de la Raspberry Pi como plataforma principal se debe a su mayor capacidad de procesamiento, lo que le permite ejecutar modelos más densos en comparación con la ESP32-CAM, que presenta limitaciones significativas en recursos de procesamiento.

- **Capacidad de Procesamiento:** La Raspberry Pi 4 cuenta con un procesador *Quad-core ARM Cortex-A72* a 1.5GHz, lo que permite la ejecución de modelos con mayor fluidez y eficiencia. Por otro lado, la ESP32-CAM utiliza un procesador *ESP32-D0WDQ6* con dos núcleos a 240MHz, lo que limita su capacidad para ejecutar modelos de detección avanzados como YOLOv8 (Bazán Andría, 2022; Raspberry Pi Ltd., 2019; Shenzhen Ai-Thinker Technology Co., Ltd., 2017).

- **Memoria y Almacenamiento:** La Raspberry Pi 4 dispone de opciones de 1GB, 2GB, 4GB y 8GB de RAM, lo que la hace adecuada para manejar modelos de aprendizaje profundo sin requerir optimizaciones agresivas. Por otro lado, la ESP32-CAM tiene solo 520KB de RAM con 4MB de Flash, lo que impide la carga de modelos pesados y restringe su funcionalidad a aplicaciones más ligeras (Salvador & Prieto, 2022).

- **Compatibilidad con Aprendizaje Profundo:** La Raspberry Pi permite la ejecución de modelos de detección, incluyendo el uso de *Intel Neural Compute Stick* y *Google Coral TPU*,

lo que mejora el rendimiento de la predicción. Sin embargo, la ESP32-CAM está diseñada para tareas básicas de reconocimiento de imágenes, pero no cuenta con los recursos necesarios para ejecutar modelos complejos como YOLOv8 (Bazán Andría, 2022).

2.11. Razones para la Selección de la Raspberry Pi

Debido a que el modelo YOLOv8 es más denso en comparación con arquitecturas más livianas, requiere una plataforma capaz de manejar cálculos matriciales y convoluciones de manera eficiente. Durante la fase de prueba, se intentó ejecutar YOLOv8 en la ESP32-CAM, pero debido a sus limitaciones en RAM y procesamiento, el modelo no pudo ser cargado en la memoria del dispositivo. Por otro lado, la Raspberry Pi mostró un mejor desempeño, logrando la ejecución del modelo con un consumo de recursos mejorado (Bazán Andría, 2022; Raspberry Pi Ltd., 2019; Shenzhen Ai-Thinker Technology Co., Ltd., 2017).

La elección de la Raspberry Pi permite implementar YOLOv8 con un balance entre costo de cómputo y portabilidad, asegurando que el modelo funcione en un entorno embebido sin comprometer la precisión ni la velocidad de detección.

Capítulo 3

Metodología

En este capítulo se describe la metodología de la investigación, detallando el proceso seguido para evaluar y comparar el desempeño del modelo YOLOv8 en su versión estándar y en la versión modificada con convoluciones separables profundas.

Luego, se describe el conjunto de datos COCO 2017, utilizado para entrenar y evaluar los modelos. Se detalla su estructura, el número de imágenes, las clases de objetos que contiene y el proceso de tratamiento de los datos. También se incluyen ejemplos de imágenes para ilustrar la diversidad y complejidad del dataset.

Posteriormente, se explican las métricas de evaluación utilizadas para medir el rendimiento de los modelos, como el Mean Average Precision (mAP) en sus diferentes variantes, Intersection over Union (IoU) y el tiempo de inferencia, entre otras. Se justifica la elección de estas métricas y su importancia en la comparación objetiva de los modelos analizados.

Finalmente, se detallan las especificaciones del modelo propuesto de YOLOv8 con convoluciones separables, describiendo los cambios realizados en la arquitectura del modelo y los ajustes en su entrenamiento y validación. Se incluyen detalles sobre los hiperparámetros utilizados, el hardware en el que se ejecutó el entrenamiento y las estrategias de optimización aplicadas. Con esta metodología, se busca establecer un marco de referencia claro y reproducible para evaluar el impacto del uso de convoluciones separables en YOLOv8, permitiendo analizar si esta modificación mejora el rendimiento del modelo sin comprometer su precisión.

3.1. Diseño de Investigación

El diseño de investigación para este proyecto de tesis es de tipo cuantitativo. El enfoque cuantitativo es adecuado debido a que se busca medir y evaluar el rendimiento del modelo YOLOv8 y YOLOv8 con convolución separables en términos de precisión, velocidad y costo de cómputo reducido. Las técnicas de recolección de datos incluyen la utilización de conjuntos de datos públicos y estandarizados, como COCO2017, para el entrenamiento y la validación del modelo.

La metodología cuantitativa se basa en la recopilación de datos numéricos y su análisis estadístico para establecer relaciones entre las variables. En este caso, las variables de interés son las métricas de rendimiento del modelo, como el Mean Average Precision (mAP) y la velocidad de predicción.

3.2. Recolección y Preparación de los Datos

La base de datos COCO (Common Objects in Context) fue creada con el objetivo de avanzar en el estado del arte del reconocimiento de objetos, proporcionando un conjunto de datos amplio y detallado. Este conjunto incluye imágenes con múltiples objetos que interactúan en entornos reales, lo que representa un desafío significativo para los modelos de detección y segmentación de objetos (Lin et al., 2014; Liu et al., 2020).

En su versión COCO2017, el conjunto de datos contiene aproximadamente 330.000 imágenes, de las cuales más de 200.000 cuentan con anotaciones. En total, el conjunto incluye cerca de 1,5 millones de instancias de objetos anotados, organizadas en 80 categorías de objetos detectables y 91 categorías de "cosas" (*stuff*), que representan elementos del entorno (COCO Consortium, 2019).

Las 80 categorías de objetos corresponden a elementos discretos y contables, como personas, vehículos y utensilios, los cuales presentan bordes claramente definidos en las imágenes. En contraste, las 91 categorías de "cosas" agrupan componentes más amplios y homogéneos del entorno, tales como agua, césped o cielo, que no poseen límites bien definidos y suelen cubrir extensas áreas dentro de la imagen.

Desde el punto de vista del número de categorías, el conjunto de datos contiene más clases de "cosas" que de objetos. Sin embargo, en términos de instancias anotadas, los objetos individuales son considerablemente más numerosos, mientras que las categorías de "cosas" aportan principalmente información del entorno.

Para el desarrollo de este trabajo, los modelos se entrenaron utilizando el conjunto de datos COCO2017, empleando 118.287 imágenes para el conjunto de entrenamiento (Train) y 5.000 imágenes para el conjunto de validación (Val), siguiendo la partición estándar utilizada en la literatura.

Durante el entrenamiento de los datos, el modelo aprende a identificar patrones y atributos de los objetos en las imágenes. Este proceso ajusta los pesos de las conexiones dentro de la red neuronal para minimizar las diferencias entre las predicciones del modelo y las etiquetas reales (Goodfellow et al., 2016).

Después del entrenamiento, el modelo evalúa el conjunto de validación para medir su capacidad de generalización. Esto asegura que el modelo no solo memorice las imágenes de entrenamiento, sino

que también sea capaz de reconocer objetos en nuevas imágenes.

3.3. Implementación del Modelo

El modelo YOLOv8 fue implementado utilizando TensorFlow y Keras. La configuración del entorno de desarrollo y entrenamiento se realizó en una computadora con las siguientes especificaciones: un procesador AMD Ryzen 7 5700X de 8 núcleos a 3.4GHz, 32GB de RAM, y una GPU NVIDIA RTX 4060 TI ideal para el procesamiento demandante de tareas de Deep Learning. Esta configuración proporcionó una capacidad de procesamiento adecuada para el manejo de grandes conjuntos de datos y modelos complejos de Deep Learning.

Dado que, en diversos repositorios y herramientas de anotación, las cajas delimitadoras (bounding boxes) se representan frecuentemente mediante las coordenadas de sus esquinas en el formato:

$$(x_0,\, y_0,\, x_1,\, y_1),$$

donde (x_0, y_0) corresponde a la esquina superior izquierda y (x_1, y_1) a la esquina inferior derecha. En ese caso, la conversión al formato basado en centro utilizado por YOLOv8 se obtiene mediante:

$$w = x_1 - x_0, \qquad h = y_1 - y_0, \qquad c_x = \frac{x_0 + x_1}{2}, \qquad c_y = \frac{y_0 + y_1}{2}.$$

Dado que los modelos de detección de objetos requieren un formato de entrada estandarizado, se realizó una transformación en la representación de las coordenadas de los cuadros delimitadores. En este trabajo, las anotaciones provienen del conjunto de datos COCO, donde las cajas se definen como:

$$(x_0,\, y_0,\, w,\, h),$$

siendo (x_0, y_0) la esquina superior izquierda y (w, h) el ancho y alto de la caja (en pixeles). Para compatibilidad con el formato utilizado por YOLOv8, se transformó a:

$$(c_x,\, c_y,\, w,\, h),$$

donde (c_x, c_y) corresponde al centro de la caja, calculado como:

$$c_x = x_0 + \frac{w}{2}, \qquad c_y = y_0 + \frac{h}{2}.$$

Esta transformación permitió aumentar la precisión de las predicciones, además de mejorar la interpretación espacial y contextual de los objetos detectados en las imágenes. La arquitectura original del modelo YOLOv8 está conformada por tres componentes fundamentales: el Backbone es el encargado de extraer características relevantes, un Neck dedicado al refinamiento y fusión de los atributos extraídos, y un Head responsable de la generación final de las predicciones. En este estudio se presenta una adaptación de la arquitectura original de YOLOv8 mediante la incorporación estratégica de

capas convolucionales separables en profundidad, tomando como base las innovaciones propuestas en las arquitecturas de MobileNet y MobileNetV2. Dicha adaptación permitió realizar detecciones más eficientes y precisas a múltiples escalas, logrando una reducción considerable en la complejidad de procesamiento, aspecto especialmente relevante para su implementación en dispositivos con recursos limitados (Howard et al., 2017; Sandler et al., 2018).

3.4. Adaptación propuesta para modelo YOLO v8 con capas convolucionales separables

Se propuso una adaptación de dicha arquitectura, reemplazando las capas convolucionales estándar basadas en `Conv2D` por capas convolucionales separables en profundidad (`SeparableConv2D`) en la arquitectura del modelo solo se modificó el módulo Head. Esta técnica, utilizada extensamente en arquitecturas eficientes como MobileNet y Xception, permite descomponer la operación convolucional en:

- Una convolución en profundidad (*depthwise convolution*), que aplica un filtro por canal de entrada de forma independiente, capturando patrones espaciales específicos.
- Una convolución punto a punto (*pointwise convolution*), que fusiona la información de los diferentes canales mediante filtros 1×1, permitiendo la combinación intercanal necesaria para la clasificación y detección.

Esta modificación redujo notablemente el número de parámetros y las operaciones por segundo (FLOPs), lo cual resulta fundamental para la implementación en entornos de hardware limitado, como dispositivos embebidos o sistemas móviles. La implementación experimental del `SeparableConv2D` en el *Head* de YOLOv8 mostró mejoras sustanciales en la rapidez computacional, manteniendo niveles competitivos de precisión en las tareas de detección (Chollet, 2017; Howard et al., 2017).

3.5. Entrenamiento y Evaluación

El proceso de entrenamiento del modelo involucró el ajuste de hiperparámetros clave como la tasa de aprendizaje, el tamaño de los lotes y el número de épocas. El modelo se entrenó utilizando el conjunto de datos COCO2017, dividiendo las imágenes en conjuntos de entrenamiento, validación y de test. Durante el entrenamiento, el modelo se encarga de aprender a identificar patrones y atributos de los objetos en las imágenes, ajustando los pesos de las conexiones dentro de la red neuronal para minimizar la diferencia entre las predicciones y las etiquetas reales comparando los 2 modelos (Goodfellow et al., 2016; Lin et al., 2014; Redmon et al., 2016).

La evaluación del modelo se realizó utilizando el conjunto de validación, midiendo métricas como el Mean Avarage Precision (mAP) a diferentes umbrales de IoU. Estas métricas permitieron evaluar

la precisión y el recall del modelo, proporcionando una visión integral de su rendimiento en diferentes escenarios de detección (LearnOpenCV, 2024; Ultralytics, 2024; V7 Labs, 2024).

En la Tabla 3.1, muestra una comparación en los parámetros del modelo YOLOv8 con convoluciones separables y convoluciones estándar.

Modelo	**Parámetros Totales**	**Parámetros Entrenables**	**Parámetros No Entrenables**
YOLOv8 Modelo Modificado Separable	1.258.715 (4,80 MB)	1.245.819 (4,75 MB)	12.896 (50,38 KB)
YOLOv8 Modelo Base Conv2D	3.991.584 (15,23 MB)	3.978.688 (15,18 MB)	12.896 (50,38 KB)

Tabla 3.1. Parámetros del modelo implementado de YOLOv8.

Como se puede observar en la Tabla 3.1, el modelo YOLOv8 con convoluciones separables presenta una reducción considerable en el número de parámetros totales y entrenables en comparación con la versión estándar basada en Conv2D. Esto implica una disminución en el consumo de memoria y, potencialmente, una mejora en la eficiencia de cómputo. Sin embargo, es importante analizar si esta reducción impacta el rendimiento del modelo en términos de precisión y velocidad de predicción.

3.5.1. Criterio de selección del número de épocas

La selección del número de épocas de entrenamiento para ambos modelos se realizará mediante un enfoque incremental, evaluando su desempeño en intervalos de 10 épocas. Este procedimiento permitirá analizar la evolución de métricas clave, tales como mAP y precisión, a lo largo del proceso de entrenamiento. Ambos modelos serán entrenados utilizando el mismo conjunto de datos de entrenamiento y evaluados sobre el mismo conjunto de validación (COCO2017), bajo métricas de desempeño idénticas, de modo que la comparación se efectuará en condiciones experimentales equivalentes. En este contexto, se espera que las diferencias que se observen puedan atribuirse principalmente a las características de las arquitecturas, más que a variaciones en los datos o en las condiciones de evaluación.

3.6. Métricas de Desempeño en Detección de Objetos

Para evaluar el desempeño de los modelos de detección de objetos, se emplean diversas métricas que permiten analizar su precisión, recall y capacidad de localización. Estas métricas son fundamentales para comparar el rendimiento de diferentes arquitecturas y configuraciones de modelos, proporcionando información sobre su eficacia en distintas condiciones. A continuación, se describen las métricas más relevantes aplicadas en este proyecto.

3.6.1. Precision

La precisión es una métrica clave en la evaluación de modelos de detección de objetos, ya que mide la proporción de predicciones correctas en relación con todas las predicciones realizadas (Ultralytics, 2024). Se define matemáticamente como :

$$Precision = \frac{TP}{TP + FP} \tag{3.1}$$

donde:

- TP (*True Positives*): Número de detecciones correctas en las que el objeto detectado coincide con el objeto verdadero.
- FP (*False Positives*): Número de detecciones incorrectas en las que el modelo predice un objeto inexistente.

Un valor alto de precisión indica que el modelo genera pocas detecciones erróneas, lo que es fundamental en aplicaciones donde los falsos positivos pueden generar consecuencias críticas, como en la seguridad y la detección de anomalías en entornos industriales (Paperspace, 2024).

3.6.2. Sensibilidad (*Recall*)

El recall o sensibilidad mide la capacidad del modelo para detectar todos los objetos presentes en una imagen. Se define matemáticamente como:

$$Recall = \frac{TP}{TP + FN} \tag{3.2}$$

donde:

- TP (*True Positives*): Número de detecciones correctas.
- FN (*False Negatives*): Número de objetos reales que el modelo no detectó.

Un valor alto de recall indica que el modelo es capaz de detectar la mayoría de los objetos presentes en la imagen, minimizando las omisiones. Sin embargo, un recall alto puede venir acompañado de un mayor número de falsos positivos, lo que afecta la precisión (Paperspace, 2024).

3.6.3. Intersection over Union (IoU)

Intersection over Union (IoU) es una métrica esencial en la evaluación de modelos de detección de objetos, ya que permite cuantificar la superposición entre una caja delimitadora predicha (*bounding box*) y la caja delimitadora real (*ground truth*). Se calcula como la relación entre el área de intersección y el área de unión de ambas cajas, lo que proporciona una medida objetiva de la precisión con la que un modelo detecta objetos (Ultralytics, 2024).

$$IoU = \frac{\text{Área}_{\text{Intersección}}}{\text{Área}_{\text{Unión}}} \tag{3.3}$$

donde:

- **Área de intersección** ($\text{Área}_{\text{Intersección}}$): Región donde la caja predicha y la caja real se superponen.
- **Área de unión** ($\text{Área}_{\text{Unión}}$): Área total combinada de ambas cajas, incluyendo la intersección.

3.6.4. Cálculo de Intersección y Unión

Dado un par de cuadros delimitadores, el área de intersección se calcula como:

$$\text{Área}_{\text{Intersección}} = \text{máx}(0, x_{\text{derecha}} - x_{\text{izquierda}}) \times \text{máx}(0, y_{\text{inferior}} - y_{\text{superior}}) \tag{3.4}$$

y el área de unión se obtiene mediante:

$$\text{Área}_{\text{Unión}} = \text{Área}_{\text{predicha}} + \text{Área}_{\text{real}} - \text{Área}_{\text{Intersección}} \tag{3.5}$$

donde:

- $x_{\text{izquierda}}, y_{\text{superior}}$ son las coordenadas de la esquina superior izquierda de cada caja.
- $x_{\text{derecha}}, y_{\text{inferior}}$ son las coordenadas de la esquina inferior derecha de cada caja.
- A_{predicho} y A_{real} representan las áreas de las cajas de predicción y real, respectivamente (Kaur & Singh, 2023).

3.6.5. Importancia del IoU en la Evaluación de Modelos

Un valor de IoU más alto indica que la predicción está mejor alineada con la caja real, lo que implica una localización más precisa del objeto detectado. En la evaluación de modelos de detección de objetos, el IoU se compara con un umbral predefinido (threshold) para determinar si una detección es considerada correcta. Algunos valores comunes de umbral son:

- **IoU = 0,50** : Un criterio permisivo donde se considera que una detección es correcta si al menos el 50 % de su área se superpone con la caja real. Se emplea en aplicaciones generales

donde una detección aproximada es suficiente, como en video vigilancia o control de tráfico (LearnOpenCV, 2024).

- **IoU = 0,75**: Un criterio más estricto, donde se requiere un mayor solapamiento entre la predicción y la verdad. Se utiliza en tareas donde la precisión espacial es más relevante, como en la inspección automatizada de productos (Paperspace, 2024).
- **IoU = 0,95** : Umbral extremadamente exigente, empleado en métricas como mAP50-95, donde se evalúa la capacidad del modelo en escenarios de detección de alta precisión. Es útil en aplicaciones como ensamblaje de circuitos electrónicos y detección de objetos pequeños o parcialmente ocultos (V7 Labs, 2024).

3.6.6. Impacto del IoU en la Detección de Objetos

IoU no sólo se emplea para evaluar modelos de detección, sino que también desempeña un papel crucial en el entrenamiento de redes neuronales especializadas en visión artificial. Durante el proceso de optimización, los algoritmos de detección de objetos, como YOLO, SSD y Faster R-CNN, utilizan IoU para:

- Ajustar la predicción de las cajas delimitadoras minimizando la pérdida durante el entrenamiento (loss function).
- Determinar si una detección es considerada un True Positive (TP) o un False Positive (FP) (Liu et al., 2020).
- Aplicar técnicas de Non-Maximum Suppression (NMS), eliminando predicciones redundantes y seleccionando la mejor caja para cada objeto detectado (Zou et al., 2019).

Un mal ajuste del umbral de IoU puede afectar significativamente el rendimiento reportado del modelo. En evaluación, fijar un umbral de IoU muy alto vuelve el criterio de coincidencia extremadamente exigente: solo se consideran verdaderos positivos aquellas predicciones cuya caja se solapa fuertemente con la caja real. En consecuencia, aunque el modelo siga generando detecciones, muchas no alcanzarán dicho solapamiento mínimo y no serán contabilizadas como aciertos, pudiendo incluso observarse muy pocos verdaderos positivos (o ninguno) si la localización no es suficientemente precisa. Por el contrario, un umbral de IoU más bajo relaja el criterio de coincidencia y aumenta la cantidad de detecciones aceptadas como correctas, aunque con menor exigencia de precisión espacial (Ultralytics, 2024).

3.6.7. Average Precision (AP)

Average Precision (AP) se calcula como el área bajo la curva de precisión-recall. Proporciona un valor único que resume el rendimiento del modelo en términos de precisión y recall a través de diferentes umbrales de confianza:

$$AP = \int_0^1 P(r)\, dr \tag{3.6}$$

donde $P(r)$ representa la curva de precisión-recall interpolada. Esta métrica es especialmente útil para comparar el rendimiento entre modelos y configuraciones, ya que integra tanto la capacidad del modelo para detectar correctamente los objetos (precisión) como su capacidad para no pasar por alto ningún objeto (recall) (LearnOpenCV, 2024; V7 Labs, 2024).

3.6.8. Mean Average Precision (mAP)

Mean Average Precision (mAP) extiende el concepto de AP al calcular el promedio de los valores de AP a través de múltiples clases de objetos. Esto es particularmente útil en escenarios de detección de múltiples clases, ya que proporciona una medición integral del rendimiento del modelo.

Se define como:

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{3.7}$$

donde N es el número total de clases en el conjunto de datos y AP_i representa la precisión promedio de la clase i. En términos simples, mAP es la media de las precisiones promedio de todas las clases del dataset, ofreciendo una visión global de cómo el modelo se desempeña en diferentes categorías. (LearnOpenCV, 2024; Paperspace, 2024)

Las categorías más utilizadas para evaluar mAP son:

- **mAP50 (Mean Average Precision a IoU 50 %)**: Es el promedio de precisiones calculado con un umbral de Intersección sobre Unión (IoU) fijo del 50 %. Este umbral es considerado "permisivo", ya que permite un mayor grado de solapamiento entre las predicciones y las anotaciones reales para que una detección sea considerada correcta. Se emplea como una métrica fundamental en la verificación de modelos de detección de objetos, ya que proporciona una referencia inicial sobre qué tan bien el modelo detecta objetos sin ser demasiado estricto en la alineación exacta de las cajas de predicción. Modelos con un mAP50 alto suelen ser eficaces para detectar objetos grandes y claramente definidos, aunque esta métrica no es suficiente para evaluar su precisión en detecciones más detalladas o en objetos pequeños y parcialmente ocultos (Paperspace, 2024).

- **mAP50-95 (Mean Average Precision en Rango de IoU 50 %-95 %)**: Esta métrica extiende la validación del modelo al calcular el promedio de precisiones en múltiples umbrales de IoU, comenzando desde 0.50 hasta 0.95, en intervalos de 0.05. A diferencia del mAP50, esta métrica proporciona una visión más completa del rendimiento del modelo bajo diferentes niveles de tolerancia al solapamiento.

$$mAP_{50-95} = \frac{1}{10N} \sum_{j=1}^{10} \sum_{i=1}^{N} AP_{i,IoU=50+5(j-1)} \tag{3.8}$$

donde:

- j representa cada uno de los 10 umbrales de IoU (desde 0,50 hasta 0,95 en incrementos de 0,05).
- N es el número total de clases en el dataset.
- $AP_{i,IoU}$ es la precisión promedio de la clase i en un umbral de IoU específico.

Un mAP50-95 alto indica que el modelo no solo detecta objetos con un solapamiento mínimo, sino que también es capaz de generar predicciones más precisas y alineadas con las anotaciones reales, lo que es crucial en aplicaciones como la detección de objetos pequeños o en entornos de alta complejidad. Debido a su naturaleza más exigente, el mAP50-95 es considerado el estándar para la evaluación de modelos de detección modernos, ya que refleja el desempeño del modelo en escenarios variados, desde detecciones más fáciles (IoU=0.50) hasta detecciones altamente precisas y restrictivas (IoU=0.95) (LearnOpenCV, 2024; Paperspace, 2024).

Capítulo 4

Resultados y Discusión

El modelo YOLOv8 es una de las arquitecturas más avanzadas en la detección de objetos, mejoradas para ofrecer un equilibrio entre precisión, velocidad de inferencia y uso eficiente de recursos de cómputo. Se basa en una red neuronal convolucional profunda (*Deep Convolutional Neural Network, DCNN*) y sigue el paradigma de detección de objetos en una sola pasada (*single-stage detector*), lo que significa que realiza la predicción de cajas delimitadoras y clasificaciones en un solo ciclo de predicción.

YOLOv8 introduce mejoras sustanciales respecto a versiones anteriores mediante la adopción de técnicas avanzadas como:

- **Backbone eficiente**: Utiliza una arquitectura mejorada con bloques CSP (*Cross Stage Partial Networks*), que reducen la redundancia de procesamiento y mejoran la extracción de características.
- **Neck mejorado**: Implementa FPN (*Feature Pyramid Networks*) y PAN (*Path Aggregation Networks*) para la fusión de características en múltiples escalas, mejorando la detección de objetos de diferentes tamaños.
- **Cabezal de detección mejorado**: Integra *Decoupled Heads* para separar la predicción de clasificación y regresión de cajas delimitadoras, optimizando el rendimiento en escenarios complejos.
- **Inferencia más rápida**: Reduce el tiempo de procesamiento mediante optimizaciones en la arquitectura y la compatibilidad con aceleradores como *TensorRT*, *ONNX Runtime* y *CUDA*.

A continuación, se presenta los análisis de los resultados obtenidos en la experimentación de dos configuraciones de YOLOv8:

- **YOLOv8-Conv2D**: Versión base que emplea convoluciones estándar (*2D Convolutional Layers*), desarrollada para lograr la máxima precisión con el costo de un mayor consumo de cálculo.

- **YOLOv8-Separable**: Variante que utiliza convoluciones separables en profundidad (*Depthwise Separable Convolutions*), lo que reduce el número de parámetros y el costo de procesamiento, permitiendo una inferencia más eficiente en dispositivos de baja potencia.

El objetivo de esta medición es comparar el rendimiento de ambas configuraciones en distintos entornos de procesamiento. La evaluación se realiza en dos plataformas representativas: un PC de escritorio con GPU dedicada y una Raspberry Pi como dispositivo embebido con recursos limitados.

- **PC de escritorio**: Entorno de alto rendimiento con GPU dedicada (*NVIDIA RTX 4060 Ti*), preparado para inferencia acelerada mediante CUDA y TensorRT.
- **Raspberry Pi**: Entorno con recursos limitados donde se analiza el rendimiento y uso eficiente de recursos de cómputo de las versiones de modelos propuestos.

Los resultados obtenidos en este análisis proporcionaron una visión detallada de la viabilidad del modelo YOLOv8 en distintas aplicaciones, permitiendo identificar posibles mejoras en la implementación y optimización del modelo para entornos con restricciones de hardware.

Para evaluar la efectividad de cada modelo, se realizaron pruebas en dos entornos de ejecución:

- **Computadora de escritorio**: Se evaluó el rendimiento tanto con una GPU dedicada, representativa de un hardware de alto desempeño, como utilizando únicamente la CPU, sin aceleración gráfica.
- **Raspberry Pi**: Se analizó la viabilidad de implementar YOLOv8 en un entorno embebido con limitaciones de memoria y procesamiento.

Este análisis se centra en métricas clave como Precision, Recall, IoU (Intersection over Union), mAP, velocidad de inferencia y consumo de memoria. Además, se exploran las diferencias en desempeño y las implicaciones prácticas de implementar cada modelo en distintos entornos.

4.1. Especificaciones del Hardware de Prueba

Para evaluar los modelos en distintos entornos de cómputo, se utilizaron los siguientes dispositivos:

Componente	**PC Escritorio**	**Raspberry Pi**
Procesador	Ryzen 7 5700X (8C/16T, 3.4GHz)	BCM2711 (ARM Cortex-A72, 1.5GHz)
GPU	RTX 4060 Ti (8GB GDDR6)	Sin GPU (CPU con NEON SIMD)
RAM	32GB DDR4 @ 3200MHz	8GB LPDDR4 @ 2133MHz
Almacenamiento	SSD NVMe 1TB (PCIe 4.0)	Tarjeta microSD 128GB
S.O.	Windows 10 Pro 64-bit	Raspberry Pi OS 64-bit
Eficiencia	Alto consumo (65W CPU, 160W GPU)	Bajo consumo (7W)
Procesamiento	Soporte CUDA/TensorRT	CPU dependiente

Tabla 4.1. Especificaciones del hardware de prueba.

Como se observa en la Tabla 4.1, la computadora de escritorio cuenta con una GPU de alto rendimiento, mientras que la Raspberry Pi depende exclusivamente de su CPU. Esta diferencia impactará en la velocidad de inferencia y la capacidad de procesamiento de YOLOv8 en cada entorno.

4.2. Métricas de Evaluación

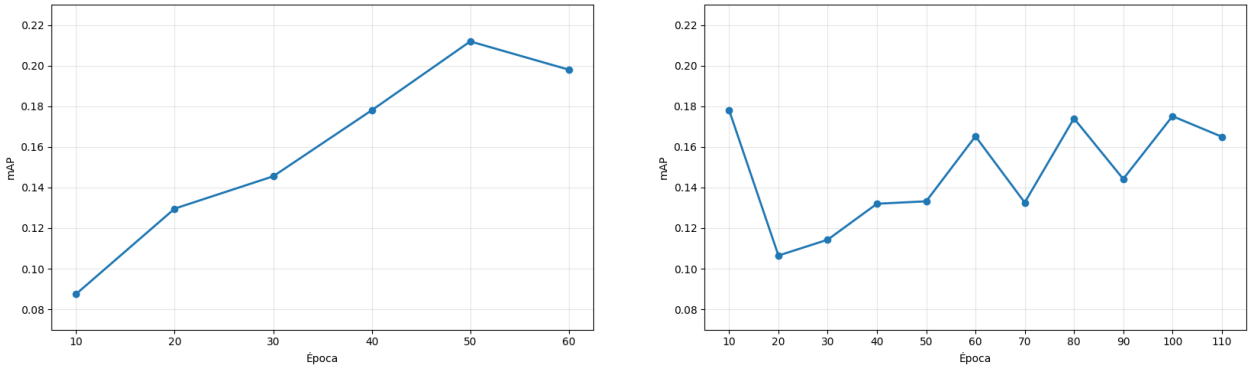
Para evaluar el desempeño de los modelos, se utilizaron las siguientes métricas:

- **Precision**: Proporción de predicciones correctas sobre el total de predicciones realizadas.
- **Recall**: Proporción de objetos correctamente detectados sobre el total de objetos en la imagen.
- **IoU (Intersection over Union)**: Mide el solapamiento entre la caja predicha y la caja real.
- **mAP**: Promedio de precisión en múltiples umbrales de IoU.
- **Velocidad de inferencia**: Tiempo medio de procesamiento por imagen en milisegundos (ms/img).
- **Consumo de memoria**: Uso de recursos en distintos dispositivos.

4.3. Resultados de la Etapa de Validación

En esta sección se presentan los resultados obtenidos en la etapa de validación para ambos modelos de detección de objetos propuestos, utilizando el conjunto COCO2017 como referencia. El objetivo es comparar el desempeño de YOLOv8-Conv2D y YOLOv8-Separable, tanto en términos de calidad de detección(mAP, IoU promedio, precisión y recall) como en la eficiencia de procesamiento (Tiempos de inferencia y su variabilidad).

En primer lugar, se analizó la evolución del *mean Average Precision* (mAP) en función del número de épocas de entrenamiento, con el propósito de justificar la selección de la configuración final empleada en cada arquitectura.



(a) (b)

Figura 4.1. Evolución de la métrica mAP por cada 10 épocas. (a) Modelo YOLOv8-Conv2D. (b) Modelo YOLOv8-Separable.

A partir de este análisis, se determinó que, para el modelo basado en convoluciones estándar observado en la Figura 4.1 a, un número de 50 épocas resulta adecuado, dado que, en torno a este punto las métricas de desempeño alcanzan una fase de estabilización, seguida de ligeras degradaciones, lo que sugiere el inicio de sobreajuste (overfitting).

Por otro lado, el modelo con convoluciones separables de la Figura 4.1 b demostró una convergencia más lenta, requiriendo un mayor número de iteraciones para aproximarse a su mejor desempeño observado, estabilizándose en torno a las 100 épocas.

También, se evalúo el comportamiento de cada modelo en distintos entornos de hardware, como CPU,GPU y Raspberry Pi con el propósito de saber el costo de cómputo en dispositivos con recursos limitados y para detección en tiempo real.

Métrica	**YOLOv8-Separable**	**YOLOv8-Conv2D**
Épocas de entrenamiento	100 Epoch	50 Epoch
mAP50	0,1751	0,2119
IoU promedio	0,8302	0,8367
Precisión	0,6734	0,6836
Recall	0,2933	0,3260
Velocidad de inferencia CPU (FPS)	2,41	2,34
Velocidad de inferencia GPU (FPS)	5,36	5,48
Velocidad de inferencia Raspberry Pi (FPS)	0,68	0,07
Tiempo promedio CPU ($\bar{t} \pm \sigma$) **(s)**	$0{,}4156 \pm 0{,}0136$	$0{,}4273 \pm 0{,}0207$
Tiempo promedio GPU ($\bar{t} \pm \sigma$) **(s)**	$0{,}1866 \pm 0{,}1671$	$0{,}1824 \pm 0{,}0169$
Tiempo promedio Raspberry Pi ($\bar{t} \pm \sigma$) **(s)**	$1{,}4754 \pm 0{,}0142$	$13{,}9322 \pm 1{,}5230$

Tabla 4.2. Resultados en validación para ambos modelos.

Los resultados de la Tabla 4.2 permitieron extraer varias conclusiones relevantes sobre el comportamiento de ambas variantes del modelo en el conjunto de COCO2017 y en los distintos entornos de hardware considerados. Los tiempos de inferencia y sus desviaciones estándar permiten cuantificar la viabilidad práctica de cada variante en distintos entornos de ejecución. YOLOv8-Separable fue entrenado por más épocas (100 vs 50), no supera el desempeño de Conv2D, lo que indica la diferencia observada explica principalmente por diferencias estructurales de capacidad representacional más que por falta de convergencia. Asimismo, la tabla permite anticipar el compromiso central del estudio: mientras que en CPU/GPU de escritorio las velocidades de inferencia son comparables, en Raspberry Pi emergen diferencias marcadas tanto en latencia promedio como en estabilidad (desviación estándar), evidenciando el costo de cálculo de Conv2D se vuelve crítico en hardware restringido.

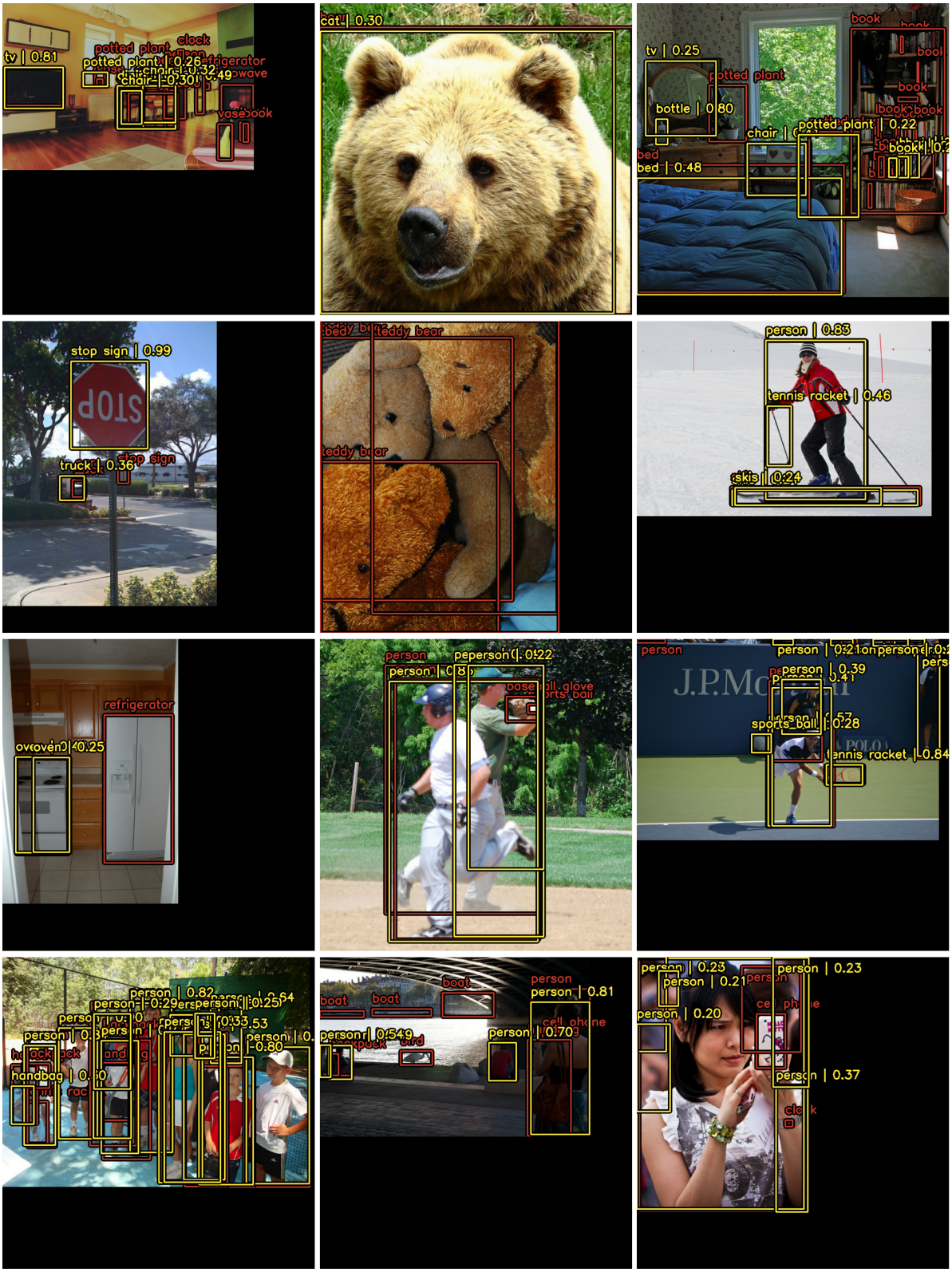


Figura 4.2. Modelo de detección con YOLOv8-Conv2D entrenado con 50 épocas.

La Figura 4.2 muestra detecciones cualitativas obtenidas por YOLOv8 Conv2D sobre COCO2017, evidenciando visualmente el ajuste de las cajas y la estabilidad de las predicciones.

- En primer lugar, se observó que la versión base YOLOv8-Conv2D obtiene el mejor desempeño en términos de precisión presenta un mAP superior (0,2119 frente a 0,1751) y mejores valores de IoU promedio, precisión y *recall*. Esto es coherente con el diseño del modelo: al utilizar capas convolucionales 2D estándar, la red dispone de una mayor capacidad de representación y, por lo tanto, puede modelar patrones más complejos en las imágenes de COCO2017. Dicha capacidad adicional se traduce en una mejor calidad de las cajas delimitadoras y una mayor tasa de verdaderos positivos, a costa de un incremento significativo en el número de parámetros y en el costo de cómputo asociado.

  El mAP obtenido por YOLOv8-Conv2D representa una mejora en detección, ya que esta métrica resume la precisión del modelo bajo umbrales de IoU; por lo tanto, su incremento implica simultáneamente una clasificación más fiable y una regresión de caja más precisa, reflejando mejor ajuste de las predicciones. Este desempeño es coherente con la mayor capacidad que entregan las convoluciones 2D estándar en el backbone y el head, favoreciendo la modelación de patrones visuales característicos de COCO2017. La evidencia cualitativa de la Figura 4.2 muestra que Conv2D mantiene detecciones más estables especialmente con objetos medianos y pequeños, con delimitadores más ajustadas. Finalmente, esta ganancia en calidad se acompaña de un mayor costo de procesamiento, pero en hardware con aceleración (CPU/GPU de escritorio) la penalización práctica es marginal, dado que las velocidades de inferencia son comparables entre modelos (2,34–2,41 FPS en CPU y 5,36–5,48 FPS en GPU), mientras que la diferencia de rendimiento se vuelve crítica principalmente en plataformas sin aceleración dedicada, como Raspberry Pi, aspecto que se profundiza al analizar la variante separable.

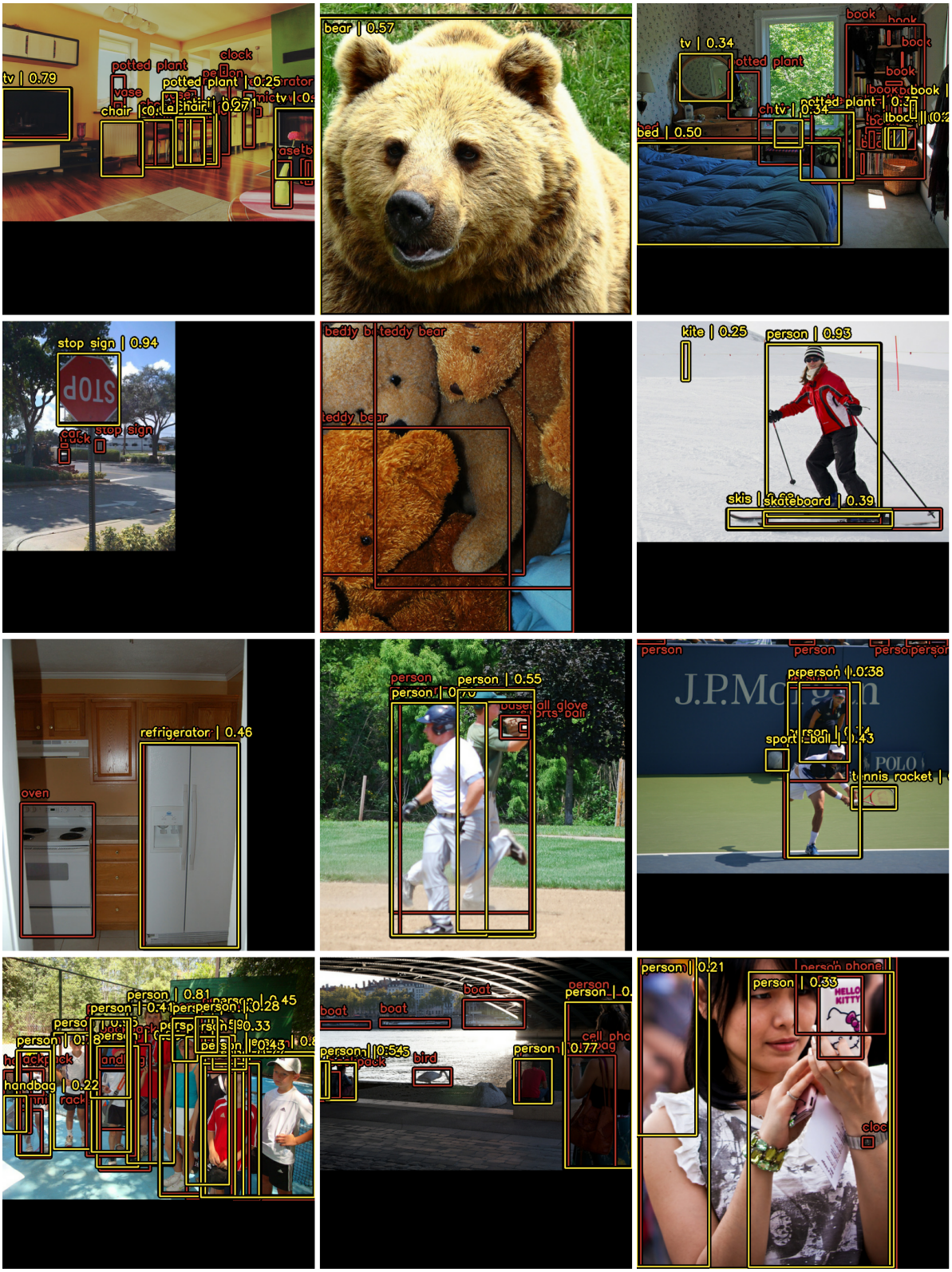


Figura 4.3. Modelo de detección con YOLOv8-Separable entrenado con 100 épocas.

- Por otro lado, YOLOv8-Separable incorporó convoluciones separables en profundidad (*depthwise separable convolutions*) como reemplazo de las convoluciones 2D estándar, incorporándolas de manera sistemática en los bloques convolucionales del detector. En términos funcionales, esta sustitución se reflejó en los módulos de fusión multiescala del Neck y en las ramas de predicción del Head, responsables de integrar características a diferentes resoluciones y de generar las predicciones finales de clases y cajas, respectivamente como se ve en la Figura 4.3. Operacionalmente, la convolución separable descompone el filtrado en una etapa por canal y una etapa de combinación entre canales mediante filtros 1×1, lo que reduce el número de parámetros y el costo computacional sin perder la capacidad de integrar información espacial.

  En la Tabla 4.2, la variante separable disminuyó el número de parámetros totales desde aproximadamente 3,99 millones a 1,25 millones, lo que equivale a una reducción cercana al 68 % respecto de la versión basada en Conv2D. En la práctica, esta compactación se traduce en menor consumo de memoria y una inferencia más eficiente, especialmente en escenarios con restricciones de cómputo; no obstante, dicha reducción de capacidad puede reflejarse en una leve disminución de las métricas de detección, evidenciando el compromiso entre eficiencia y desempeño.

- Un detalle importante del experimento es el costo de entrenamiento de ambos modelos. Debido a la densidad del modelo, el tamaño del conjunto COCO2017 (118.287 imágenes de entrenamiento y 5000 de validación) y a la resolución de entrada (640 X 640), por cada época completada, incluyendo la fase de entrenamiento y la evaluación sobre el conjunto de validación, tardó entre 10 y 12 horas en la GPU utilizada. En la práctica, esto implica que entrenar 50 épocas del modelo YOLOv8-Conv2D y 100 épocas del modelo YOLOv8-Separable supuso varias semanas de cómputo efectivo para poder entrenar el modelo en las mejores condiciones, quedándonos con el mAP más alto que se adapta mejor a la imagen. Esta limitación temporal condicionó el número de épocas que fue posible ejecutar y, en consecuencia, restringió la exploración sistemática de hiperparámetros (tamaño del lote, tasa de aprendizaje, entre otros). Es esperable que el modelo de mayor capacidad (Conv2D) alcance valores de mAP en menores épocas, mientras que el modelo separable requiera más iteraciones de entrenamiento para compensar parcialmente la reducción de parámetros.

- Respecto a la velocidad de inferencia, los resultados mostraron comportamientos distintos según en el entorno de ejecución. En la computadora de escritorio, donde se cuenta con una GPU dedicada, ambos modelos presentan valores de FPS muy similares tanto como CPU como en GPU. Esto sugiere que, en este escenario, el cuello de botella está principalmente relacionado con la carga de datos, las transferencias memoria-GPU y el propio entorno de ejecución de TensorFlow, más que con el número exacto de parámetros del modelo. La presencia de una GPU moderna (RTX4060 Ti) permite amortiguar el aumento de complejidad de YOLOv8-Conv2D, de modo que la ventaja teórica en FLOPs de la versión separable no se traduce en una ganancia de velocidad tan marcada.

- La situación cambió de forma notable en la Raspberry Pi, donde el procesamiento depende exclusivamente de la CPU ARM Cortex-A72 y no existe aceleración por GPU. en este entorno, YOLOv8-Separable alcanza 0,68 FPS frente a los 0,07 de YOLOv8-Conv2D, lo que supone casi un orden de magnitud de mejora en la velocidad de inferencia. El tiempo promedio de procesamiento por imagen pasó de 13,93 s en el modelo Conv2D a 1,48 s en el modelo separable, acompañado de una reducción significativa en la desviación estándar de los tiempos (de 1,5230 s a 0,0142 s). Este comportamiento indica que la versión separable no es solo más rápida, sino también más estable en la latencia cuadro a cuadro, lo cual es crítico en aplicaciones de tiempo sobre hardware embebido.

- Las diferencias de estabilidad también estuvieron vinculadas a la forma en que fue necesario adaptar los modelos a TensorFlow Lite para ejecutarlos en la Raspberry Pi. Durante el proceso de exportación, se detectó que ciertas operaciones en particular *DepthwiseConv2dNative* no estaban soportadas por los núcleos nativos de TFLite, lo que obligó a habilitar *Select TF Ops* y a reestructurar el grafo del modelo. Para lograr una ejecución correcta fue necesario separar explícitamente la parte de inferencia pura de las etapas asociadas al codificador de etiquetas y al proceso de *Non-Maximum-Supression* (NMS), manteniendo únicamente las capas imprescindibles para la predicción en el modelo convertido. Esta reestructuración permitió desplegar la arquitectura entrenada en la Raspberry Pi, pero también implicó que una fracción del grafo se ejecutó como operaciones *flex*, que no siempre pueden beneficiarse plenamente de las optimizaciones internas de TFLite y de la librería XNNPACK. El impacto de la limitación es más evidente en el modelo Conv2D, más pesado y con mayor ancho de banda de memoria, lo que explica tanto sus tiempos de inferencia más altos como la mayor variabilidad observada en las mediciones.

- En resumen, los resultados de validación confirman el compromiso entre precisión y costo/eficiencia de cómputo. YOLOv8-Conv2D ofrece el mejor rendimiento, por lo que resulta más adecuado para escenarios en lo que se dispone de hardware de alto rendimiento y la prioridad es maximizar la calidad de las detecciones. Por su parte, YOLOv8-Separable sacrifico una fracción de dicha precisión, pero redujo de manera importante el número de parámetros, el consumo de memoria y el tiempo de inferencia, especialmente en plataformas embebidas como la Raspberry Pi. Bajo estas condiciones el modelo con convoluciones separable es una alternativa viable para sistemas de anti colisión y otras aplicaciones de visión por computador que deban operar de forma continua en dispositivos de bajo consumo, donde la eficiencia y la estabilidad de la latencia son tan importantes como la precisión absoluta del modelo.

Capítulo 5

Conclusiones

En relación con el objetivo comparativo de la tesis, los resultados de validación demostraron que el modelo de YOLOv8-Conv2D presentó una ventaja en calidad de detección sobre COCO2017 (mAP 0,2119 vs 0,1751) a pesar de haber sido entrenado con menos épocas, lo que refuerza su mayor capacidad representacional que impacta directamente la precisión y la localización. En cambio, en Raspberry Pi se evidencia el efecto esperado de la sustitución de Conv2D a Separable: la variante separable reduce drásticamente la latencia y estabiliza la inferencia (0,68 vs 0,07 FPS; 13,93 s vs 1,48 s), confirmando la relación entre desempeño y viabilidad de despliegue en entornos embebidos

Si bien los resultados validan el compromiso precisión-eficiencia entre ambas variantes, su generalización debe considerar los siguientes alcances. La estimación del desempeño se restringió al conjunto COCO2017 y a dos plataformas de ejecución (computador de escritorio y Raspberry Pi), por lo que la magnitud podría variar en dominios con distribuciones distintas o en hardware embebido con aceleración dedicada. Las configuraciones de entrenamiento difirieron en número de épocas (50 para Conv2D y 100 para Separable), lo que introduce una asimetría experimental adicional; no obstante, la brecha persistente sugiere que el factor dominante corresponde a diferencias arquitecturales de capacidad representacional. Finalmente, las mediciones en Raspberry Pi dependieron del flujo de conversión y despliegue en TensorFlow Lite, lo cual puede afectar las latencias absolutas, por lo que los tiempos deben interpretarse principalmente relativos entre modelos.

El rendimiento en el modelo basado en convoluciones estándar (*Conv2D*) presentó una mayor precisión en la detección de objetos, pero con un costo de cálculo considerablemente más alto, reflejado en mayores tiempos de predicción y un uso más intensivo de recursos de procesamiento y memoria. Esto lo convierte en una alternativa menos viable para dispositivos embebidos o con restricciones de procesamiento.

Por el contrario, el modelo implementado con convoluciones separables en profundidad mostró una ligera reducción en la precisión respecto al modelo estándar, pero logró mantener un desempeño suficientemente robusto para ser considerado dentro del umbral de *precisión aceptable* en aplicaciones de visión por computadora. Esta categoría hace referencia a un equilibrio en el cual la exactitud del modelo es suficientemente buena para cumplir con los objetivos de la tarea, sin requerir una carga computacional elevada.

Uno de los desafíos encontrados fue la adaptación de los modelos a la Raspberry Pi, ya que la ejecución en este dispositivo presentó restricciones de capacidad de cómputo y en el consumo energético. Sin embargo, con el uso de técnicas como la cuantización de pesos y la poda de parámetros, es posible mejorar la eficiencia sin sacrificar drásticamente el rendimiento del modelo.

En función de los resultados obtenidos, se concluyó que la elección de la arquitectura del modelo debe estar alineada con los requisitos específicos de la aplicación. Es decir, si se busca priorizar la precisión y se dispone de hardware con alto poder de cálculo, la versión con Conv2D es una opción viable. En cambio, si la implementación se orienta a dispositivos embebidos o móviles con restricciones de memoria y procesamiento, el uso de convoluciones separables se presenta como una alternativa efectiva.

Los resultados en distintos entornos de hardware confirmaron que la mejor precisión observada en el modelo de Conv2D se mantiene en plataformas con aceleración, donde las diferencias de latencia resultan marginales; sin embargo, dicha ganancia implica un costo de cómputo significativo en dispositivos reducidos. En Raspberry Pi, la configuración con convoluciones separables no solo reduce el tiempo promedio de procesamiento por imagen, sino que también disminuye su variabilidad, lo que se traduce en mayor estabilidad cuadro a cuadro y en una ventaja en aplicaciones en tiempo real. Este patrón se vincula directamente con la menor carga paramétrica y de memoria del modelo separable, así como con las limitaciones prácticas identificadas durante la exportación a TensorFlow Lite, las cuales afectan con mayor intensidad el grafo estándar. En consecuencia, la elección entre ambos modelos debe fundamentarse en el contexto de despliegue: Conv2D para maximizar la calidad en la infraestructura de alto rendimiento, y el modelo Separable para sistemas embebidos donde el uso eficiente de recursos de cómputo y una latencia estable constituyen requisitos operacionales.

En conclusión, esta investigación demuestra que la selección adecuada de la arquitectura del modelo de detección de objetos puede mejorar significativamente la eficiencia de procesamiento sin comprometer en exceso la precisión.

5.1. Futuras líneas de Investigación

Como futura línea de investigación, sería interesante explorar estrategias de optimización avanzadas, como la combinación de técnicas de compresión de modelos con aceleración por hardware (TPUs o NPUs), y evaluar el impacto en distintos conjuntos de datos y escenarios de aplicación. También sería interesante estudiar la adaptación de estos modelos en entornos de producción para validar su desempeño en situaciones reales, midiendo métricas clave como la latencia de detección y el consumo energético.

5.1.1. Evaluación del Rendimiento en Diferentes Hardware

En cuanto al aspecto de estudiar en mayor profundidad cómo afecta el hardware a las métricas de comparación de detección de objetos, se sugieren los siguientes dispositivos:

- **Google Edge TPU**: Diseñada para acelerar tareas de visión por computadora con bajo consumo energético, esta plataforma permitiría evaluar la eficiencia de modelos optimizados en dispositivos IoT.
- **NVIDIA Jetson**: Un entorno ampliamente utilizado en robótica y automatización, donde la optimización del rendimiento y la latencia de inferencia son fundamentales.
- **FPGAs y NPUs**: Estas arquitecturas permiten configuraciones específicas para acelerar la inferencia de modelos de visión computacional, lo que puede ofrecer mejoras en eficiencia y consumo energético (Jouppi et al., 2017).

Bibliografía

BASF Chemicals. (2023). *AI-based product classification in chemical manufacturing*. https://www.basf.com/global/en/who-we-are/digitalization/artificial-intelligence

Bazán Andría, J. (2022). *Implementación en una Raspberry Pi de una red neuronal convolucional con propiedades de tolerancia a fallos*. https://riunet.upv.es/handle/10251/187563

BMW Manufacturing. (2023). *AI-assisted quality control in BMW factories*. https://www.press.bmwgroup.com/global/article/detail/T0449729EN/artificial-intelligence-as-a-quality-booster?language=en

Bochkovskiy, A., Wang, C., & Liao, H. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. *CoRR*, *abs/2004.10934*. https://arxiv.org/abs/2004.10934

Changi Airport Group. (2023). *AI-based security surveillance at Changi Airport*. https://www.changiairport.com/en/corporate/our-media-hub/publications/changi-journeys.changi-airport-harnesses-ai-for-smarter-security-screening.2023.airport-that-never-sleeps.html#:~:text=The%20new%20system%20uses%20AI,and%203D%20X%2Dray%20images.

Chollet, F. (2017). Xception: Deep Learning with Depthwise Separable Convolutions. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1800-1807. https://doi.org/10.1109/CVPR.2017.195

COCO Consortium. (2019). *COCO: Common objects in context*. https://cocodataset.org/#detection-2019

Foxconn Technology Group. (2023). *Automated defect detection in electronics manufacturing*. https://e.huawei.com/id/case-studies/industries/manufacturing/ai-quality-inspection-2023

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. http://www.deeplearningbook.org

Han, S., Mao, H., & Dally, W. J. (2016). *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. arXiv: 1510.00149 [cs.CV]. https://arxiv.org/abs/1510.00149

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *37*(9), 1904-1916. https://doi.org/10.1109/TPAMI.2015.2389824

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778. https://doi.org/10.1109/CVPR.2016.90

Hendrycks, D., & Gimpel, K. (2023). *Gaussian Error Linear Units (GELUs)*. arXiv: 1606.08415 [cs.LG]. https://arxiv.org/abs/1606.08415

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *CoRR*, *abs/1704.04861*. http://arxiv.org/abs/1704.04861

Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *CoRR*, *abs/1502.03167*. http://arxiv.org/abs/1502.03167

Jocher, G., Chaurasia, A., Stoken, A., Borovec, J., NanoCode012, Kwon, Y., Michael, K., Xie, T., Fang, J., imyhxy, Lorna, Zeng, Y., Wong, C., V, A., Montes, D., Wang, Z., Fati, C., Nadar, J., Laughing, ... Jain, M. (2022). *ultralytics/yolov5: v7.0 - YOLOv5 SOTA Realtime Instance Segmentation (v7.0)* (v7.0). https://doi.org/10.5281/zenodo.7347926

Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., Boyle, R., Cantin, P.-l., Chao, C., Clark, C., Coriell, J., Daley, M., Dau, M., Dean, J., Gelb, B., ... Yoon, D. H. (2017). In-Datacenter Performance Analysis of a Tensor Processing Unit. *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 1-12. https://doi.org/10.1145/3079856.3080246

Kaur, R., & Singh, S. (2023). A comprehensive review of object detection with deep learning. *Digital Signal Processing*, *132*, 103812. https://doi.org/10.1016/j.dsp.2022.103812

LearnOpenCV. (2024). *Mean average precision (mAP) in object detection*. https://learnopencv.com/mean-average-precision-map-object-detection/

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, *86*(11), 2278-2324. https://doi.org/10.1109/5.726791

Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *Proceedings of the European Conference on Computer Vision (ECCV)*, 740-755. https://arxiv.org/abs/1405.0312

Liu, L., Ouyang, W., Wang, X., Fieguth, P., Chen, J., Liu, X., & Pietikäinen, M. (2020). Deep learning for generic object detection: A survey. *International Journal of Computer Vision*, *128*(2), 261-318. https://doi.org/10.1007/s11263-019-01247-4

Long, J., Shelhamer, E., & Darrell, T. (2015). Fully convolutional networks for semantic segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3431-3440. https://doi.org/10.1109/CVPR.2015.7298965

Meituan. (2022). *YOLOv6: Efficient object detection*. https://github.com/meituan/YOLOv6

Murel, J., & Kavlakoglu, E. (2023). *What is object detection?* Consultado el 24 de noviembre de 2025, desde https://www.ibm.com/think/topics/object-detection

Neubeck, A., & Van Gool, L. (2006). Efficient Non-Maximum Suppression. *18th International Conference on Pattern Recognition (ICPR'06)*, *3*, 850-855. https://doi.org/10.1109/ICPR.2006.479

NFL Security Team. (2023). *AI-powered crowd monitoring at Super Bowl LVII*. https://www.sportsbusinessjournal.com/Daily/Issues/2023/02/13/Technology/sb-attention/

Odena, A., Dumoulin, V., & Olah, C. (2016). Deconvolution and Checkerboard Artifacts. *Distill*. https://doi.org/10.23915/distill.00003

Paperspace. (2024). *Mean average precision (mAP) explained*. https://blog.paperspace.com/mean-average-precision/

Raspberry Pi Ltd. (2019). *Raspberry Pi 4 Model B Datasheet* (inf. téc.). Raspberry Pi Ltd. https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788. https://doi.org/10.1109/CVPR.2016.91

Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, Faster, Stronger. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 6517-6525. https://doi.org/10.1109/CVPR.2017.690

Ring. (2023). *Smart home security with AI-powered object detection*. https://www.aboutamazon.com/news/devices/ring-camera-4k-home-security

Salvador, R., & Prieto, C. (2022, octubre). *Sistema de seguridad con reconocimiento facial en módulo ESP32*. https://doi.org/10.52948/mare.v4i1.684

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4510-4520. https://doi.org/10.1109/CVPR.2018.00474

Shenzhen Ai-Thinker Technology Co., Ltd. (2017). *ESP32-CAM Wi-Fi+BT SoC Module Datasheet (V1.0)* (inf. téc.). Ai-Thinker. https://gzhls.at/blob/ldb/9/5/1/c/fa48c01c284d3f9299f1b92f2286d6c6314a.pdf

Terven, J., Córdova-Esparza, D.-M., & Romero-González, J.-A. (2023). A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, *5*(4), 1680-1716. https://doi.org/10.3390/make5040083

Ultralytics. (2024). *YOLO performance metrics*. https://docs.ultralytics.com/es/guides/yolo-performance-metrics/

V7 Labs. (2024). *Mean average precision (mAP) explained: Everything you need to know*. https://www.v7labs.com/blog/mean-average-precision/

Varghese, R., & M., S. (2024). YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness. *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, 1-6. https://doi.org/10.1109/ADICS58448.2024.10533619

Wang, C., Liao, H. M., Yeh, I., Wu, Y., Chen, P., & Hsieh, J. (2019). CSPNet: A New Backbone that can Enhance Learning Capability of CNN. *CoRR*, *abs/1911.11929*. http://arxiv.org/abs/1911.11929

Zhang, Y., & Wang, L. (2024). YOLOv7-DSE: An efficient safety equipment detection network. *IAENG International Journal of Computer Science*, *51*(6), 572-581. https://www.iaeng.org/IJCS/issues_v51/issue_6/IJCS_51_6_02.pdf

Zhao, Z.-Q., Zheng, P., Xu, S.-T., & Wu, X. (2019). Object Detection With Deep Learning: A Review. *IEEE Transactions on Neural Networks and Learning Systems*, *30*(11), 3212-3232. https://doi.org/10.1109/TNNLS.2018.2876865

Zou, Z., Shi, Z., Guo, Y., & Ye, J. (2019). Object Detection in 20 Years: A Survey. *arXiv preprint arXiv:1905.05055*. https://arxiv.org/abs/1905.05055